

第10章 差错控制编码

10.1 差错控制编码的基本原理

在实际的通信过程中，由于信道传输特性不理想以及加性噪声的影响，收到的信息不可避免地会发生错误。对于一个通信系统而言，其主要质量指标是通信的有效性和通信的可靠性。在数字通信系统中，编码器分为两大类：一类是信源编码，另一类是信道编码。信源编码的目的是解决有效性问题，前面已经做过讨论。信道编码（即差错控制编码）的目的是提高通信的可靠性。以上两类编码都属于信息论的主要内容。1948年香农

（Shannon）发表的“通信的数学理论”论文奠定了数字通信的理论基础。随着数字通信技术的普及，高速数据网的出现与发展，大容量、高速传输的数字通信可靠性问题已越来越受到人们的重视。根据不同的通信业务，用户对误码率有不同的要求。当一般的信道不能满足要求时，就要采用差错控制编码技术。如何降低误码率，提高数字通信的可靠性已成为数字通信的重要研究课题之一。差错控制编码正是为此目的而发展起来的一项技术。它产生于20世纪50年代，发展到70年代走向成熟。

10.1.1 引起误码的原因及降低误码的常用方法

数字信号在实际通信的过程中不可避免地会发生错误。这是因为，一方面通信系统的特性并非完全理想，数字信号波形通过这样的通信系统时会产生波形失真，因而在接收端判决时会产生判决错误。这种干扰称为乘性干扰。另一方面，信道中的噪声也会产生干扰，这种干扰随机地与信号

叠加，使信号波形产生失真，也会引起判决错误，这种干扰称为加性干扰。对于乘性干扰可以通过均衡器来消除码间串扰的影响。随机加性干扰通常由信道产生。根据随机加性干扰的不同特点，相应的信道可以分为三类：随机信道、突发信道、混合信道。

(1) 随机信道：在这种信道中，错码的出现是互不相关、统计独立的。比如，当信道中的随机加性干扰主要是高斯分布的白噪声时，引起的错码就具有这种性质。

(2) 突发信道：错码的出现是前后相关的。当错码出现时，会在短时间内有一连串的错码，而该时间过后又有较长的时间无错码。造成突发错码的原因是信道中存在随机的强突发脉冲干扰，比如，闪电、电焊、电火花干扰等。当信道中的随机加性干扰主要是随机强突发脉冲干扰时，称为突发信道。

(3) 混合信道：以上两种随机干扰都存在，产生的错码既有随机错码又有突发错码，称这种信道为混合信道。

了解了不同的信道类型，就可以根据不同的信道特点采用相应的差错控制方式。

在介绍差错控制方式之前，首先了解一下降低误码、提高数字通信可靠性的几种途径。根据实际通信系统对其可靠性的要求，可以用以下不同的方法提高通信的可靠性。

(1) 适当增加发送信号功率。增大信号的功率，即提高输入端的信噪比，可以减少信道中随机加性干扰对信号的影响，降低误码率，提高数字通信的可靠性。但是发送信号功率由于受到设备和环境条件的限制而不能无限增大。比如，空间探测器上的发射机由于其体积和重量受到限制，其发射功率不可能无限增大。所以这种方法在实际中受到一定限制。

(2) 选择抗噪声性能好的调制解调方式。比如，在数字系统中 PSK 方式

比 ASK 方式的误码率要小得多。所以在实际中多采用 PSK 方式，而很少采用 ASK 方式。

(3) 采用最佳接收。最佳接收可以使数字通信的误码率达到最小。接收机中的滤波器可以是带通滤波器也可以是匹配滤波器。在数字通信中采用匹配滤波接收，可以最大限度地抑制白噪声，从而在判决时刻达到最大输出信噪比，降低误码率。

(4) 采用差错控制编码。它通过一定的编码和译码方法，采用前向纠错或检错重传技术，可以自动纠正传输错误。对于不同类型的信道，应采用不同的差错控制方式。这是本章要介绍的主要内容。

10.1.2 差错控制编码的基本方法及差错控制方式

差错控制编码的基本方法是：在发送端，给要传送的信息序列按照事先约定好的规律增加一些码元，称之为监督码元。使信息序列与监督码元之间具有某种相关性。在接收端，按照事先约定好的规律检验信息序列与监督码元之间的关系。如果数字信号在传输的过程中发生了错误，则信息序列与监督码元之间的关系就被破坏。根据被破坏的情况，就可以发现错误或纠正错误。有些编码方式能够发现错误，有些编码方式不仅能够发现错误，而且还能纠正错误。

对于不同类型的信道，要采用不同的差错控制方式。不同的差错控制编码也要与相应的差错控制方式配合使用。常用的差错控制方式有以下几种。

1. ARQ (Automatic Repeat Request) 方式

ARQ 方式又称检错重发或自动请求重发方式。其系统组成如图 10-1-1 所示。

编码器采用能够发现错误的编码方式。发端发送出可以发现错误的码字，经过传输，到接收端译码后，如果没有发现错误，则输出。如果发现

错误，则自动请求发端重发，直到正确接收到码字为止。可见自动请求重发方式需要双向信道。一个是信息传输的正向信道，另一个是传送请求重发指令的反向信道。自动请求重发方式发出的码字只需能够发现错误即可，所以需要的监督码元很少。这种方式对各种信道都能进行检测，并且解码电路很简单。但是自动请求重发方式由于需要双向信道，所以不能用于单向信道系统和网络中的广播系统。当干扰很大时，由于不断有错码，需要不断重发，会使通信效率降低，甚至不能通信使系统出现死锁。因为重发时会产生延迟，所以自动请求重发不适合速率要求较高的实时系统。

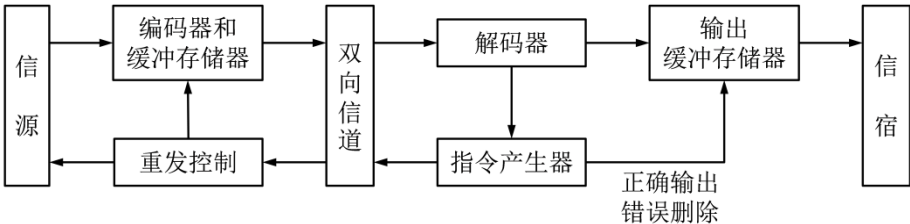


图 10-1-1 ARQ 系统组成

常用的自动请求重发系统有三种类型：停止等待 ARQ、连续 ARQ、选择重发 ARQ。

(1) 停止等待 ARQ

停止等待 ARQ 是最简单的 ARQ 系统，也称为空闲 RQ (idle-RQ)。这种系统每发送一个分组就停止发送，等待接收端的应答信号。收到接收端的确认应答后，再发送下一个分组。如果收到的是否认应答，则重发原分组。

停止等待 ARQ 操作简单，所需缓冲器容量小。但是在等待应答时花费了时间，没有传输数据，所以传输效率很低。停止等待 ARQ 不适合工作在高速传输系统中，也不适合于信号往返延迟较大的场合。

(2) 连续 ARQ

连续 ARQ 工作在全双工方式，需要有一定的缓冲器容量。这种系统两

端同时发送信息，发端连续发送数据，并接收应答信号，收端连续接收数据并发送应答信号。发送的每个分组可以分配一个顺序号。如果发端收到的都是确认应答，则发端按顺序号连续发送分组。如果发端收到了否认应答，则发端取出否认应答所含的序号，并返回至收端希望收到的分组的顺序号，从该顺序号开始，按顺序连续发送分组。所以连续 ARQ 重发时要将错误分组及其以后的分组全部重发。比如发端连续发送分组，并不断收到确认应答。如果传输中出现错误，假设当发完 9 号分组后收到否认应答，该否认应答表示 5 号分组有错，希望收到 5 号分组。则发端返回至 5 号分组，并按顺序从 5 号分组开始重发 5 号分组及其以后的分组数据。

连续 ARQ 在信道好、误码率低时传输效率很高。但在信道差、误码率高、信号往返延迟较大时，由于重发的分组太多，使其传输效率较低，不宜使用。

(3) 选择重发 ARQ

选择重发 ARQ 是由连续 ARQ 发展而来的。工作在全双工方式，需要有较大的缓冲器容量。其工作过程类似于连续 ARQ。但在发送端重发时，不是将错误分组及其以后的分组全部重发，而是仅重发出错的分组。这样就要求对分组的顺序进行管理，需要复杂的控制和大容量的缓冲器，增加了系统的复杂程度。但与其它 ARQ 系统相比，选择重发 ARQ 在信道差、误码率高时性能较好。

2. 反馈校验方式

反馈校验方式是发送端一边发送码字，一边将发送的码字在发端缓冲存储。当接收端收到码字后，立即将接收到的码字返回发送端。发送端将返回的码字与发端缓冲存储器中相应的码字比较，若发现与发送码不同，即认为产生了错误，就重发上一次的码字，直到发送端校验正确为止。利用这种方式进行差错控制其设备简单，但要求双向信道，并且传输效率很

低。

3. FEC (Forward Error Control) 方式

FEC 方式即采用前向纠错的差错控制方式。编码器采用能够纠正错误的编码方式。发送端发出的码字不仅能够发现错误, 而且能够纠正错误。在接收端译码后, 若没有错误则直接输出。若有错误, 则在接收端自动纠正后, 再输出。这种方法不需要反向信道, 实时性好, 传输效率高。但纠错编译码方法复杂, 所以所需设备较复杂。

4. HEC (Hybrid Error Control) 方式

HEC 方式即混合纠错检错方式。它是将 ARQ 方式和前向纠错方式结合使用。一般的纠错编码能够检错和纠错的位数都是有限的。比如一种纠错编码它能纠正一个码字内的两位错, 检出三位错。当码字中出现两位以下错码时, 它能自动纠正错码。当码字中出现两位以上错码时, 它不能自动纠正。所以在传输错码较少时, 采用前向纠错方式, 自动纠正错码。在错码较多时, 采用 ARQ 方式自动请求重发。这种方式综合了 ARQ 方式和前向纠错方式的优点。既有利于提高通信的可靠性, 又有利于提高系统的传输效率。

10.1.3 有扰信道的编码定理

香农有扰信道的编码定理表明: 在有扰信道中只要信息的传输速率 R 小于信道容量 C , 总可以找一种编码方法, 使信息以任意小的差错概率通过信道传送到接收端, 即误码率 P_e 可以任意小, 而且传输速率 R 可以接近信道容量 C 。但若 $R > C$, 在传输过程中必定带来不可纠正错误, 不存在使差错概率任意小的编码。上述误码率 $P_e = e^{-nE(R)}$ 。式中 n 为编码的码字长度 (简称码长), $E(R)$ 为误码指数。 $E(R)$ 与 R 、 C 有关, 它与 R 、 C 的关系可用曲线表示, 如图 10-1-2 所示。

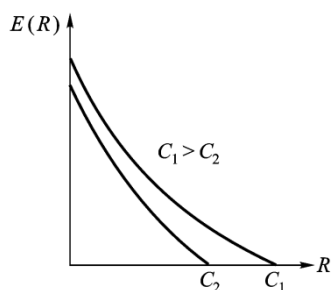


图 10-1-2 $E(R)$ 与 R 的关系

从公式 $P_e = e^{-nE(R)}$ 及 $E(R)$ 与 R 、 C 的关系曲线可以看出，要提高抗干扰能力，减小误码率 P_e 可以有两种途径：

(1) 在码长 n 及信息的传输速率 R 一定时，为减小 P_e 可以增加信道容量 C 。由图 10-1-2 可见， $E(R)$ 随信道容量 C 的增加而增大。由公式可见，

误码率 P_e 随 $E(R)$ 的增大而指数减小。从信道容量公式 $C = B \log_2 \left(1 + \frac{S}{N} \right)$ 看

出：要增加信道容量 C ，可以通过增加信号功率 S 和系统带宽 B 来实现。

(2) 在信道容量 C 及信息的传输速率 R 一定的情况下，增加码长 n 可以使误码率 P_e 指数减小。这从公式可直接看出。

(3) 香农有扰信道的编码定理本身并未给出具体的纠错编码方法，但它为信道编码奠定了理论基础。从理论上指出了信道编码的发展方向。

10.1.4 重复编码的例子

假设要发送天气预报消息，而且天气只用两种状态表示：有雨，无雨。采用第一种编码方法，将有雨编为 **1**，无雨编为 **0**。两个码字只有一位差别。在信道噪声干扰下，如果 **1** 误传为 **0** 或者 **0** 误传为 **1**，在接收端都不可能识别是否有错，只能得到错误的结论，因为所收到的码字都是许用码（预先约定好的码字）。这种情况只能收到错误的天气预报。

可以采用另一种编码方法，将有雨编为 **11**，无雨编为 **00**。即给第一种

编码再加一位重复码元，这样两个码字之间的差别有两位。增加的重复码元称为监督码元。而原有的码元称为信息码元。该码字总的码长为 2。在信道噪声干扰下，如果干扰使码字中仅一位发生错误，即接收端出现 **10** 或 **01** 码。这时接收端会发现在预先约定好的码字（许用码）中并不存在这样的码字，称为禁用码（通信双方未约定好的码字）。出现了禁用码，接收端就可以判断必然是传输中出现了错误。但是发端原来所发的码字到底是 **11** 还是 **00**，收端还是难以判断。可见这种两个码字之间的差别有两位时，可以检测出一位错，但不能纠正错误。如果两位同时出错，即 **11** 误传为 **00** 或者 **00** 误传为 **11**，也不可能识别是否有错。

还可以采用第三种编码方法，将有雨编为 **111**，无雨编为 **000**。即给第一种编码再加两位重复监督码元。**111** 和 **000** 这两个码字之间有三位不同。该码字总的码长为 3。传输过程中当码字受到干扰，将 **111** 或 **000** 错误地传输为 **001**, **010**, **011**, **100**, **101**, **110** 时，接收端都可以很容易地判断是传输出现了错误，因为这些码字都不是许用码，而是禁用码。这些错误的码字可能是错一位造成的（如 **000** 错传为 **001**），也可能是错两位造成的（如 **111** 错传为 **001**）。所以它可以发现两位错。如果在二进制对称信道（BSC）中，转移概率（误码率） $P_e < \frac{1}{2}$ ，则根据最大似然译码准则可以纠正一个错误，做出正确的判决。

最大似然译码准则是：在收到码字 r 的条件下，计算 2^k 个许用码的条件概率 $P(r|C_L)$ ，其中 C_L 为许用码。若某个许用码的条件概率 $P(r|C_L)$ 最大，则认为收到的就是该 C_L 码字。 $P(r|C_L)$ 可用下式计算

$$P(r|C_L) = \prod_{i=0}^{n-1} p\left(\frac{r_i}{C_{Li}}\right) \quad (10-1-1)$$

式中， r_i 为接收码字的第 i 位元素； C_{Li} 为许用码字 C_L 的第 i 位元素。

显然，若接收码元出错，即 $r_i \neq C_{Li}$ 时，概率 $P(r_i/C_{Li}) = P_e$ ；当码元接收

正确，即 $r_i = C_{Li}$ 时，概率 $P(r_i/C_{Li}) = 1 - P_e$ 。令 d 为接收的码字与许用码 C_L 之间不同的位数（即出错位的位数），则式（10-1-1）可以写成

$$P(r|C_L) = (1 - P_e)^{n-d} P_e^d \quad (10-1-2)$$

可见，由于 $P_e < \frac{1}{2}$ ，故 $P(r|C_L)$ 随 d 单调下降。因此， d 越小， $P(r|C_L)$ 越大。 $P(r|C_L)$ 越大表明接收到的码字 r 越象码字 C_L ，而不象其它许用码，因为传输中码字错的位数多比错的位数少出现的概率更小。比如上面所举的例子中，收到的码字为 **001**，**010**，**100** 时根据上述准则可认为是 **000** 错一位造成的，判决为 **000**；当收到的码字为 **011**，**101**，**110** 时，则可认为是 **111** 错一位造成的，直接判决为 **111**。可见这种编码可纠正一位错。从上可知最大似然译码就是看接收到的码字最象哪个许用码，即判决为该许用码。

由上例可见，码字（用 C 表示）由许多码元组成，码字中码元的个数称为码长（用 n 表示），如码字 $C = (c_{n-1} \ c_{n-2} \ \dots \ c_0)$ ，其中 c_{n-1} 、 c_{n-2} 、 $\dots$ 、 c_0 表示码字中的码元。由多个许用码字构成一组码，称为码组。如上述的码组 **111**、**000**，由两个许用码字构成。其中码字 **111** 的码长 $n = 3$ 。从上述例子还可以看出，两个码字之间不同位数的多少直接决定着其检错纠错能力的大小。两个码字之间不同的位数越多，其检错纠错能力越强。

10.1.5 码间距离 d 及检错纠错能力

1. 码间距离 d

码间距离是一个码组中任意两个码字之间对应位上码元取值不同的位数，用 d 表示。码间距离（code distances）简称码距，又称汉明距离。对二元码，码间距离 d 可用下式计算

$$d(c_i, c_j) = \sum_{p=0}^{n-1} (c_{ip} \oplus c_{jp}) \quad (10-1-3)$$

即码间距离 d 等于两个码字对应位模 2 相加后 **1** 的个数。如上述例子中三

位重复码的码距 $d(111, 000)=3$ 。

另如两个码字 c_i 、 c_j 分别为 **101110** 和 **101011**，则码距可按下式计算

$$\begin{array}{r} c_i: \quad 101110 \\ \oplus \quad c_j: 101011 \\ \hline 000101 \end{array}$$

故 $d(c_i, c_j) = 2$ 。

在一个码组中各码字之间的距离不一定都相等，有的大，有的小。称码组中最小的码距为最小码间距离，用 d_0 表示。由上述重复编码的例子可知，两个码字之间不同的位数越多，其检错纠错能力越强。即码间距离越大，其检错纠错能力越强。所以一个码组的最小码间距离 d_0 决定了该码组的检错纠错能力。

对于三位编码的码间距离，可用三维几何空间来说明。三位编码的码字共有 $2^3=8$ 个，可用三维几何空间立方体的 8 个顶点来表示，如图 10-1-3 所示。码字之间的距离可用对应两顶点间沿立方体各棱行走的最短几何距离来示意。由图可见，对上述重复编码的例子其码组只有 **111**、**000** 两个许用码字，从 **111** 到 **000** 要经过三条边，显然它们之间的距离为 $d=3$ 。同样，对于多位编码的码间距离，可用多维空间来说明。

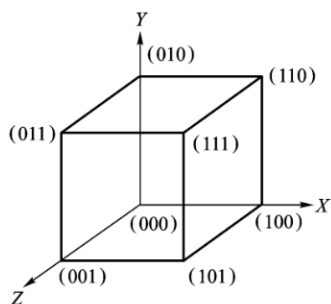


图 10-1-3 码间距离的几何意义

2. 最小码间距离 d_0 与检错纠错能力的关系

(1) 当码组仅用于检测错误时，若要求检测 e 个错误，则最小码距为

$$d_0 \geq e+1 \quad (10-1-4)$$

这可由图 10-1-4(a)来说明。图中 A 为一个码字，B 为另一个码字。若码字 A 有两位错，则 A 变为以 A 为圆心以 2 为半径的圆上某点。只要最小码距不小于 3，在半径为 2 的圆上及圆内就不会有其它许用码，因而可判断出其出错，所以能检测错码的位数等于 2。即最小码距为 d_0 ，将能检测 (d_0-1) 个错误。若要检测 e 个错误，则必须满足 $d_0 \geq e+1$ 的要求。

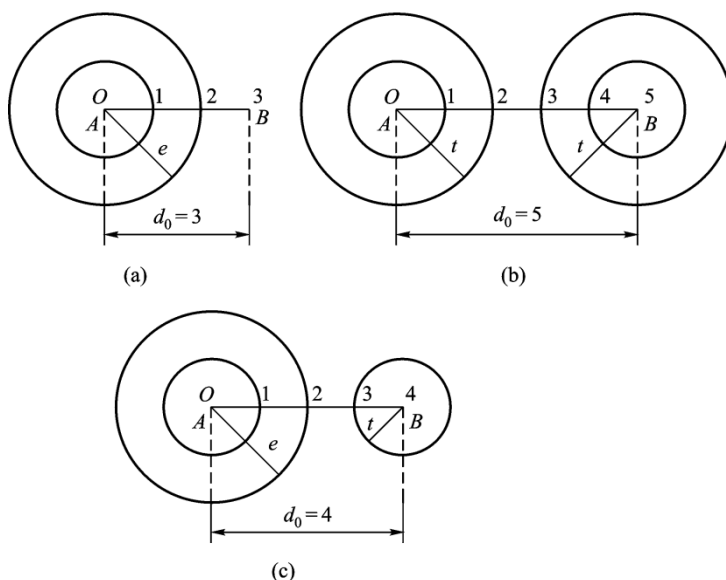


图 10-1-4 码间距离与检错纠错能力关系

(2) 当码组仅用于纠正错误时，为纠正 t 个错误，要求最小码距为

$$d_0 \geq 2t+1 \quad (10-1-5)$$

这可用图 10-1-4(b)来说明。当码字 A 发生两位错时，则落在以 A 为圆心以 2 为半径的圆上某点。码字 B 有两位错时，则落在以 B 为圆心以 2 为半径的圆上某点。只要这两个圆不重叠、不相交，就能区分判别出左边圆上的为码字 A，右边圆上的为码字 B。可见能纠正两位错码，它要求的最小码距为 5。所以纠正 t 个错误，必须满足 $d_0 \geq 2t+1$ 的要求。

(3) 当码组既要检错，又要纠错时，为纠正 t 个错误，同时检测 e 个错误，则要求的最小码距为

$$d_0 \geq e+t+1 \quad (e > t) \quad (10-1-6)$$

这可用图 10-1-4(c)来说明。当码字 A 出现 e 个错误，将落在以 A 为圆心以 e 为半径的圆上某点。码字 B 出现 t 个错误时，将落在以 B 为圆心以 t 为半径的圆上某点。要纠正码字 B 的错误，同时又能检出码字 A 的错误，就要求 A 的大圆和 B 的小圆不相交、不重叠。即 A 和 B 之间的距离要大于 $(e+t)$ ，也即最小码距 $d_0 \geq e+t+1$ 。

3. 差错控制编码的效果

对于随机错误的情况，假设误码率为 P , $P \ll 1$ 。在码长为 n 的码字中刚好发生 r 个错误的概率为 $P_n(r) = C_n^r P^r (1-P)^{n-r}$ 。当 $n=7$, $P=10^{-3}$ 时，有

$$P_7(1) \approx 7 \times 10^{-3}$$

$$P_7(2) \approx 2.1 \times 10^{-5}$$

$$P_7(3) \approx 3.5 \times 10^{-8}$$

可见，若采用差错控制编码纠正 1~2 位错，就可以使误码概率下降很多。其效果显著。

4. 纠错编码的分类

纠错编码的分类可用图 10-1-5 所示。

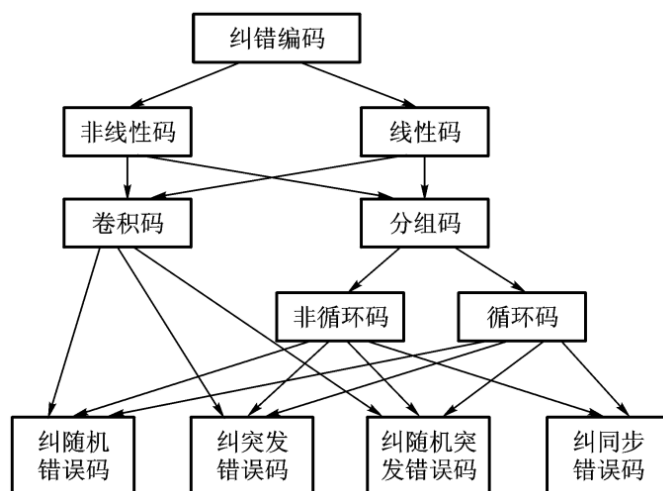


图 10-1-5 纠错编码分类示意图

纠错编码按照其实现的功能可以分为检错码和纠错码。能够发现错误但不能纠正错误的码称为检错码。在译码时不仅能够发现错误，而且能够自动纠正错误，通过译码产生正确的码字，称为纠错码。

按信息码元与监督码元的关系，可以分为线性码和非线性码。线性码中信息码元与监督码元之间存在着线性关系。目前实际应用的一般都是线性码。当信息码元与监督码元之间为非线性关系时，称为非线性码。

纠错编码还可分为分组码和卷积码。分组码是把信息码分为许多小段，即分组，每组信息码附加若干监督码。监督码元仅与本分组中的信息码元有关。而在卷积码中，监督码元不仅与本组中的信息码有关，而且与前后若干分组中的信息码元有关。

另外，按照码字的结构特点是否具有循环性，还可分为循环码和非循环码。按照码元的取值可分为二进制码和多进制码。按照纠错的类别分为纠随机错误的码和纠突发错误的码。

5. 编码效率

在分组码编码中，加入的监督位越多检错纠错能力越强。但这会使总的码长增加，即要传输的位数增加，使编码的效率降低。设码字的信息码元个数为 k ，监督码元个数为 r ，总码元的个数（即总的码长）为

$$n=k+r \quad (10-1-7)$$

编码效率 η 是指码字的信息码元个数 k 与总的码长 n 的比值，即

$$\eta = \frac{k}{n} = \frac{n-r}{n} = 1 - \frac{r}{n} \quad (10-1-8)$$

可见监督位越多，编码效率越低。

10.2 常用的简单编码

在数字通信技术的发展过程中，人们通过不断地摸索和实践总结，创造了多种简单有效的编码方法。下面介绍几种实用的简单编码。

10.2.1 奇偶监督码

奇偶监督码（也称奇偶校验码）广泛应用于计算机数据传输中。

奇偶监督码的编码规则是在每个分组的信息位后增加监督位，无论信息位有多少位，监督位只有一位。奇偶监督码可分为奇监督码和偶监督码两种，这两种编码的工作原理和检错能力相同。

偶监督码是在信息位后增加一位监督位，使码字中 **1** 的数目为偶数。满足如下关系

$$c_{n-1} \oplus c_{n-2} \oplus L \oplus c_1 \oplus c_0 = 0 \quad (10-2-1)$$

其中 $c_{n-1}c_{n-2}L c_1$ 为信息位， c_0 为监督位。式（10-2-1）即为偶监督码的监督关系，也称为校验方程。由校验方程可见它能检测出奇数个错。当发生奇数个错时，异或结果为 **1**，不满足校验方程，可判断该码字有错。当发生偶数个错时，异或结果为 **0**，满足校验方程，虽然码字有错，但检测不到，所以这种码不能检测偶数个错。

在编码的过程中监督位是由信息位产生的，该编码监督位的产生方程为

$$c_0 = c_{n-1} \oplus c_{n-2} \oplus L \oplus c_1 \quad (10-2-2)$$

奇监督码是在信息位后增加一位监督位，使码字中 **1** 的数目为奇数。其校验方程为

$$c_{n-1} \oplus c_{n-2} \oplus L \oplus c_1 \oplus c_0 = 1 \quad (10-2-3)$$

奇监督码也只能检测出奇数个错。其监督位的产生方程为

$$c_0 = c_{n-1} \oplus c_{n-2} \oplus L \oplus c_1 \oplus 1 \quad (10-2-4)$$

奇偶监督码的编码效率 η 较高，尤其是当码长 n 较大时这一特点更为明显。在 n 值很大时编码效率趋近于 1，即

$$\eta = 1 - \frac{r}{n} = 1 - \frac{1}{n} \quad n \rightarrow \infty \quad \eta \rightarrow 1 \quad (10-2-5)$$

奇偶监督码的编码和译码在电路上也很容易实现，采用简单的数字电路即可完成。

10.2.2 二维奇偶监督码

二维奇偶监督码也称方阵码、行列监督码或水平—垂直奇偶监督码。其编码方法是把 m 个信息码字排列成一个方阵，每个码字构成方阵的一行，在每一行的最后按奇偶监督规则增加一位水平监督位，按行检测每行的奇数个错，对行实行监督。然后再按列的方向每列增加一位垂直监督位（包括行监督位的列），按列检测每列的奇数个错，对列实行监督，构成水平—垂直奇偶监督码。如图 10-2-1 所示。

$$\begin{array}{cccc|c}
 C_{n-1}^1 & C_{n-2}^1 & \dots & C_1^1 & C_0^1 \\
 C_{n-1}^2 & C_{n-2}^2 & \dots & C_1^2 & C_0^2 \\
 \dots\dots\dots & & & & \\
 C_{n-1}^m & C_{n-2}^m & \dots & C_1^m & C_0^m \\
 \hline
 C_{n-1}^0 & C_{n-2}^0 & \dots & C_1^0 & C_0^0
 \end{array}$$

图 10-2-1 二维奇偶监督码的图示

二维奇偶监督码不仅可以检测每行的奇数个错和每列的奇数个错，而且行列交叉可以检测每行或每列的偶数个错。但当发生的错误为刚好构成矩形的四个错码时，则不能检测出错误。例如当图中 C_{n-1}^2 、 C_1^2 、 C_{n-1}^m 、 C_1^m 同时出错时，按行为偶数个错，按列也为偶数个错，均不能检测出来。

二维奇偶监督码在某些情况下还可以进行纠错。当出现短时的突发干扰时，误码出现非常集中，可能只有一行出现奇数个错码。此时，按行检测可以判断出错在那一行，按列检测可以确定该行的那一列发生了错误，

行列交叉可以判断错误的位置，即可纠错。此外，此种编码的效率较高。

10.2.3 恒比码

恒比码的特点是每个许用码字含有相同数目的 **1**。在码字中 **1** 与 **0** 的个数之比是恒定的，所以称之为恒比码。一个码字中 **1** 的个数称为该码字的码重，因此恒比码又称为等重码。对于某种特定的恒比码，当码长确定后，其 **1** 的个数就确定了。所以在检测中只要计算 **1** 的个数就可以确定是否发生错误。恒比码多用于电传机中。我国电传机传输汉字采用的是“5 中取 3”恒比码，其码长为 5，码字中 **1** 的个数为 3。这种码我国称为保护电码。码长为 5 的二进制数共有 32 种组合，选择其中含有 3 个 **1** 的组合作为许用码，所以“5 中取 3”恒比码许用码的个数为 $C_5^3=5!/(3!2!)=10$ 个。这十个码字刚好用来表示十个阿拉伯数字 0~9，再用这十个阿拉伯数字拼成汉字进行电码传输。我国的保护电码比国际用五单位码具有更强的抗干扰能力。表 10-2-1 列出了这两种编码。

表 10-2-1 我国的保护电码与国际电码

阿拉伯数字	保护电码	国际电码	阿拉伯数字	保护电码	国际电码
0	01101	01101	5	00111	00001
1	01011	11101	6	10101	10101
2	11001	11001	7	11100	11100
3	10110	10000	8	01110	01100
4	11010	01010	9	10011	00011

国际无线电报码广泛采用“7 中取 3”恒比码。每个码字中有三个 **1**，其码长为 7，是从 128 个码字中选出 $C_7^3=7!/(3!4!)=35$ 个作为许用码，用于代表 26 个英文字母及其它符号。国际通用的“7 中取 3”恒比码如表 10-2-2

所示。

表 10-2-2 国际通用的“7 中取 3”恒比码

字 符	码	字 符	码
A -	0 0 1 1 0 1 0	S '	0 1 0 1 0 1 0
B ?	0 0 1 1 0 0 1	T 5	1 0 0 0 1 0 1
C :	1 0 0 1 1 0 0	U 7	0 1 1 0 0 1 0
D +	0 0 1 1 1 0 0	V =	1 0 0 1 0 0 1
E 3	0 1 1 1 0 0 0	W 2	0 1 0 0 1 0 1
F %	0 0 1 0 0 1 1	X /	0 0 1 0 1 1 0
G	1 1 0 0 0 0 1	Y 6	0 0 1 0 1 0 1
H	1 0 1 0 0 1 0	Z +	0 1 1 0 0 0 1
I 8	1 1 1 0 0 0 0	回 行	1 0 0 0 0 1 1
J	0 1 0 0 0 1 1	换 行	1 0 1 1 0 0 0
K (	0 0 0 1 0 1 1	字母键	0 1 0 0 1 1 0
L)	1 1 0 0 0 1 0	数字键	0 0 0 1 1 1 0
M .	1 0 1 0 0 0 1	间 隔	1 1 0 1 0 0 0
N ,	1 0 1 0 1 0 0	(不用)	0 0 0 0 1 1 1
O 9	1 0 0 0 1 1 0	RQ	0 1 1 0 1 0 0
P 0	1 0 0 1 0 1 0	α	0 1 0 1 0 0 1
Q 1	0 0 0 1 1 0 1	β	0 1 0 1 1 0 0
R 4	1 1 0 0 1 0 0		

恒比码的主要优点是简单，它适用于电传机及其它产生固定字符的键盘设备中。

10.3 线性分组码

10.3.1 线性分组码的概念

线性分组码是将信息序列分为每 k 位一组的信息序列段，每一信息序列段按照一定规律添加 r 个监督码元，构成总码长为 $n=k+r$ 的分组码，记为 (n, k) 。在分组码中监督码元仅与本分组中的信息码元有关，即只监督本码字中的信息码元。当监督码元与信息码元之间为线性关系时，即监督码元与信息码元之间的关系可用模 2 加代数方程描述时，称其为线性分组码。线性码是建立在代数学中群论的基础上，线性码的各许用码的集合构成代数学中的群，又称为群码。

表 10-3-1 示出了线性分组码的一种结构。码字的前一部分是连续 k 个信息码元，后一部分是连续 r 个监督码元，具有这种结构的线性分组码称为系统码。不按这种结构顺序排列的线性分组码称为非系统码。

表 10-3-1 线性分组码的一种结构

$c_{n-1} \ c_{n-2} \ c_{n-3} \ \cdots \ c_r$	$c_{r-1} \ c_{r-2} \ c_{r-3} \ \cdots \ c_1 \ c_0$
k 个信息码元	r 个监督码元
总码长 $n=k+r$	

上节讲述的重复码，奇偶校验码均为线性分组码。如重复码组 **111, 000** 为 $(3, 1)$ 分组码， $n=3$ ， $k=1$ ， $r=2$ 。

线性分组码 (n, k) ，总码长为 n ，可以表示 2^n 个状态，即共有 2^n 个码字。 k 位信息位可以表示 2^k 个状态，即共有 2^k 个码字是表示信息的。称这 2^k 个码字为许用码。这 2^k 个码字中的监督位都是由监督关系确定的。当 k 个信息位的值确定后， r 个监督位的值也就确定了。余下的 $(2^n - 2^k)$ 个码

字不表示信息，为禁用码。在接收端解码校验时，利用监督关系判断。当接收端出现禁用码时，即可判断出错。

10.3.2 线性分组码的构成及监督矩阵

设要构成的线性分组码为（7，3）码，码长 $n=7$ ，信息位长 $k=3$ ，监督位长 $r=n-k=4$ 。码字矢量 $\mathbf{C}=[C_6C_5C_4C_3C_2C_1C_0]$ ，其中 $C_6C_5C_4$ 为信息码元， $C_3C_2C_1C_0$ 为监督码元。假定监督码元与信息码元的关系由下列线性方程组决定

$$\left. \begin{aligned} C_3 &= C_6 \oplus C_4 \\ C_2 &= C_6 \oplus C_5 \oplus C_4 \\ C_1 &= C_6 \oplus C_5 \\ C_0 &= C_5 \oplus C_4 \end{aligned} \right\} \quad (10-3-1)$$

上式移项后可得四个监督关系式（该方程组在二元有限域上求解，系数取值为 0 或 1）

$$\left. \begin{aligned} C_6 \oplus C_4 \oplus C_3 &= 0 \\ C_6 \oplus C_5 \oplus C_4 \oplus C_2 &= 0 \\ C_6 \oplus C_5 \oplus C_1 &= 0 \\ C_5 \oplus C_4 \oplus C_0 &= 0 \end{aligned} \right\} \quad (10-3-2)$$

按照监督关系式（10-3-1）或（10-3-2）可以确定（7，3）码的许用码共有 $2^3=8$ 个。它是从 $2^7=128$ 种组合中选出的，如表 10-3-2 所示。

表 10-3-2 （7，3）码许用码

信息码	许 用 码						
	C6	C5	C4	C3	C2	C1	C0
000	0	0	0	0	0	0	0
001	0	0	1	1	1	0	1
010	0	1	0	0	1	1	1

011	0	1	1	1	0	1	0
100	1	0	0	1	1	1	0
101	1	0	1	0	0	1	1
110	1	1	0	1	0	0	1
111	1	1	1	0	1	0	0

该 (7, 3) 码的全部许用码字都必须受到监督方程组式 (10-3-2) 的监督 and 检验, 因此又称该方程组为一致监督方程。

将式 (10-3-2) 中的零系数项补上, 写出系数可得到下式

$$\left. \begin{aligned} 1 \cdot c_6 \oplus 0 \cdot c_5 \oplus 1 \cdot c_4 \oplus 1 \cdot c_3 \oplus 0 \cdot c_2 \oplus 0 \cdot c_1 \oplus 0 \cdot c_0 &= 0 \\ 1 \cdot c_6 \oplus 1 \cdot c_5 \oplus 1 \cdot c_4 \oplus 0 \cdot c_3 \oplus 1 \cdot c_2 \oplus 0 \cdot c_1 \oplus 0 \cdot c_0 &= 0 \\ 1 \cdot c_6 \oplus 1 \cdot c_5 \oplus 0 \cdot c_4 \oplus 0 \cdot c_3 \oplus 0 \cdot c_2 \oplus 1 \cdot c_1 \oplus 0 \cdot c_0 &= 0 \\ 0 \cdot c_6 \oplus 1 \cdot c_5 \oplus 1 \cdot c_4 \oplus 0 \cdot c_3 \oplus 0 \cdot c_2 \oplus 0 \cdot c_1 \oplus 1 \cdot c_0 &= 0 \end{aligned} \right\} \quad (10-3-3)$$

把式(10-3-3)写成如下矩阵形式

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_6 \\ c_5 \\ c_4 \\ c_3 \\ c_2 \\ c_1 \\ c_0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (10-3-4)$$

并将(10-3-1)式也写成矩阵形式

$$\begin{bmatrix} c_3 \\ c_2 \\ c_1 \\ c_0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_6 \\ c_5 \\ c_4 \end{bmatrix} = \mathbf{P} \begin{bmatrix} c_6 \\ c_5 \\ c_4 \end{bmatrix} \quad (10-3-5)$$

令(10-3-4)式的系数矩阵为 $\mathbf{H}$

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (10-3-6)$$

则(10-3-4)式可简写为

$$HC^T = O^T \quad \text{或} \quad CH^T = O \quad (10-3-7)$$

其中行矩阵 $C=[c_6c_5c_4c_3c_2c_1c_0]$ 为码字矢量, $O=[0000]$ 。 C^T 、 O^T 、 H^T 分别为 C 、 O 、 H 的转置矩阵。

系数矩阵 H 决定着信息码元和监督码元之间的监督关系, 称之为线性分组码的一致监督矩阵或称一致校验矩阵。对 (n, k) 线性分组码, H 为 r 行 n 列的矩阵, 它表示了 r 个监督关系式。 H 确定则监督关系确定。 H 是产生监督码元 $c_3c_2c_1c_0$ 的依据, 也是检错纠错的依据。由(10-3-4)式和(10-3-5)式可见: H 确定后, 已知信息码即可确定监督码, 其产生的许用码满足(10-3-7)式。因此所有的许用码必然都满足(10-3-7)式。所以在接收端就可以用 H 矩阵来校验接收的码字是否是许用码。设接收到的码字为 R , 将 R 代入(10-3-7)式, 若 $RH^T = O$, 则说明 R 为许用码。若 $RH^T \neq O$, 则表明不是许用码, 即可以判断为误码。

进一步分析 H , 利用矩阵分块的方法, H 可写为

$$H = \begin{bmatrix} 1 & 0 & 1 & | & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & | & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & | & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & | & 0 & 0 & 0 & 1 \end{bmatrix} = [P \mid I_4] \quad (10-3-8)$$

其中, P 为式 (10-3-5) 中的 P , I_4 为 4 阶单位矩阵, 即

$$P = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (10-3-9)$$

对于 (n, k) 分组码, $r = n - k$, $\mathbf{H}$ 可写为

$$\mathbf{H} = [\mathbf{P} \mid \mathbf{I}_r] \quad (10-3-10)$$

其中 $\mathbf{P}$ 为 $r \times k$ 阶矩阵, $\mathbf{I}_r$ 为 r 阶单位矩阵。具有这种形式的 $\mathbf{H}$ 矩阵称为典型监督矩阵。典型监督矩阵 $\mathbf{H}$ 中的每一行都是彼此独立的, 即线性不相关, 故不能从几个方程的组合推出方程组的另一个方程。应当注意各种码的 $\mathbf{H}$ 矩阵不一定是典型阵, 只有系统码才符合。

10.3.3 线性分组码的生成矩阵

生成矩阵是在已知信息码元时确定相应的许用码字 $\mathbf{C} = [c_6 c_5 c_4 c_3 c_2 c_1 c_0]$ 的矩阵。由(10-3-1)式已经可以产生监督码元 $c_3 c_2 c_1 c_0$, 只要在其中添上信息码元的方程即可得出许用码字。如下式

$$\left. \begin{aligned} c_6 &= c_6 \\ c_5 &= c_5 \\ c_4 &= c_4 \\ c_3 &= c_6 \oplus c_4 \\ c_2 &= c_6 \oplus c_5 \oplus c_4 \\ c_1 &= c_6 \oplus c_5 \\ c_0 &= c_5 \oplus c_4 \end{aligned} \right\} \quad (10-3-11)$$

将式(10-3-11)写成矩阵形式

$$\mathbf{C}^T = \begin{bmatrix} c_6 \\ c_5 \\ c_4 \\ c_3 \\ c_2 \\ c_1 \\ c_0 \end{bmatrix} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{1} & \mathbf{0} & \mathbf{1} \\ \mathbf{1} & \mathbf{1} & \mathbf{1} \\ \mathbf{1} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{1} \end{bmatrix} \begin{bmatrix} c_6 \\ c_5 \\ c_4 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_k \\ \mathbf{P} \end{bmatrix} \begin{bmatrix} c_6 \\ c_5 \\ c_4 \end{bmatrix} \quad (10-3-12)$$

对式(10-3-12)取转置, 得

$$\mathbf{C} = \begin{bmatrix} c_6 & c_5 & c_4 \end{bmatrix} \begin{bmatrix} \mathbf{1001110} \\ \mathbf{0100111} \\ \mathbf{0011101} \end{bmatrix} = \begin{bmatrix} c_6 & c_5 & c_4 \end{bmatrix} \mathbf{G} \quad (10-3-13)$$

在式(10-3-13)中

$$\mathbf{G} = \begin{bmatrix} \mathbf{1001110} \\ \mathbf{0100111} \\ \mathbf{0011101} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_k & \mathbf{Q} \end{bmatrix} \quad (10-3-14)$$

其中 $\mathbf{Q} = \begin{bmatrix} \mathbf{1110} \\ \mathbf{0111} \\ \mathbf{1101} \end{bmatrix}$ 且 $\mathbf{Q} = \mathbf{P}^T$ 或 $\mathbf{P} = \mathbf{Q}^T$ (10-3-15)

矩阵 $\mathbf{G}$ 称为线性分组码的生成矩阵。 $\mathbf{G}$ 为 $k \times n$ 阶矩阵，行数为信息位的个数，列数为码字的长度。已知矩阵 $\mathbf{G}$ 和信息码后，由式（10-3-13）可生成许用码字 $\mathbf{C}$ 。

由式（10-3-14）可见 $\mathbf{G} = [\mathbf{I}_k | \mathbf{Q}]$ ，其中 $\mathbf{I}_k$ 为 k 阶单位矩阵。这种形式的生成矩阵 $\mathbf{G}$ 是典型生成矩阵。同样，典型生成矩阵的各行也是线性无关的。由典型生成矩阵得出的码字 $\mathbf{C}$ 是信息位在前，监督位在后的系统码。实际上 $\mathbf{G}$ 中的每一行都是一个许用码字。 $\mathbf{G}$ 中的第一行、第二行、第三行分别是信息位为[100]、[010]、[001]时计算出的许用码字。

由式(10-3-14)可知 $\mathbf{Q} = \mathbf{P}^T$ (或 $\mathbf{P} = \mathbf{Q}^T$)。由于 $\mathbf{H} = [\mathbf{P} | \mathbf{I}_r]$ ， $\mathbf{G} = [\mathbf{I}_k | \mathbf{Q}]$ ，所以当已知监督矩阵 $\mathbf{H}$ 和生成矩阵 $\mathbf{G}$ 中任意一个时，另一个即可确定，其监督关系和它所对应的分组码也就确定了。

在线性分组码中，任意两个许用码字对应位模 2 相加还是此码组中的一个码字。所以线性分组码具有封闭性。对 (n, k) 线性分组码来说，其信息位长为 k ，共有 2^k 个不同组合的信息码。设 $\mathbf{C}_{ix}$ 、 $\mathbf{C}_{jx}$ 为其中两个信息码，由式（10-3-13）可算出它们所对应的许用码字 $\mathbf{C}_i$ ， $\mathbf{C}_j$ 为

$$C_i = C_{ix} G \quad C_j = C_{jx} G$$

所以

$$C_i \oplus C_j = C_{ix} G \oplus C_{jx} G = (C_{ix} \oplus C_{jx}) G = C_l G \quad (10-3-16)$$

式(10-3-16)中 C_l 还是一个 k 位的二元序列, 它必然是 2^k 个不同组合的信息码中的一个, 所以 $C_i \oplus C_j$ 必然是生成矩阵为 G 的线性分组码中的一个许用码字。

(n, k) 线性分组码 A 的生成矩阵 G 的每一行都是码组 A 的一个许用码字, 它一定满足 H 矩阵所确定的 r 个监督关系。如果把 G 当作另一个码组 B 的监督矩阵, H 当作码组 B 的生成矩阵, 则码组 B 为 $(n, n-k)$ 线性分组码, H 的每一行一定满足 G 矩阵所确定的 k 个监督关系。这样的码组 A 和码组 B 称为对偶码。

线性分组码的最小距离 $d_{\min}$ 等于该码的最小重量 (除全 0 码字外)。即

$$d_{\min} = d_{\min}(C_i, C_j) = W_{\min} \quad (10-3-17)$$

由于线性分组码具有封闭性, 从码距的定义可知任意两个许用码字之间的距离必然是另一个许用码字的重量。所以该码的最小重量 (除全 0 码字外) 必然是该线性分组码的最小距离。

对线性分组码, 由监督矩阵 H 中线性不相关的列数可以得到线性分组码中最小码距的上界, 为

$$d_{\min} \leq n - k + 1 = r + 1 \quad (10-3-18)$$

由 $CH^T = 0$ 可见, 当 C 取最小重量的码字, 即 C 中 1 的个数为 $W_{\min}$ 时, 则得到 H 中最小相关的列的数目, 即 H 中小于或等于 $W_{\min} - 1$ 列是线性独立, 不相关的。 H 为 $n-k$ 行矩阵, 其最大的秩为 $n-k$ 。根据矩阵的性质, 则 H 中最大不相关的列数小于或等于 H 的秩, 可得 $W_{\min} - 1 \leq n - k$, 即 $d_{\min} - 1 \leq n - k$, 或写为 $d_{\min} \leq n - k + 1 = r + 1$ 。当上式取等号时, $d_{\min} = n - k + 1 = r + 1$,

称为最大可辨距离。

以上介绍了线性分组码的构成原理及其监督矩阵和生成矩阵,下面讨论线性分组码的译码。

10.3.4 线性分组码的伴随式和检错纠错能力

设发端发出的许用码为 $\mathbf{C}=[c_{n-1}c_{n-2} \dots c_1c_0]$, 它符合 $\mathbf{CH}^T = \mathbf{O}$ 。经过信道传输后, 假设收端收到的码字为 $\mathbf{R}=[r_{n-1}r_{n-2} \dots r_1r_0]$ 。如果 $\mathbf{R}=\mathbf{C}$, 把 $\mathbf{R}$ 代入(10-3-7)式计算, 则 $\mathbf{RH}^T = \mathbf{O}$ 判断为正确。但由于传输误差 $\mathbf{R}$ 与 $\mathbf{C}$ 不一定相同, 其误差为

$$\mathbf{E}=\mathbf{R}-\mathbf{C}=[e_{n-1}e_{n-2} \dots e_1e_0] \text{ (模 2) 或 } \mathbf{R}=\mathbf{C} \oplus \mathbf{E} \quad (10-3-19)$$

式中, $\mathbf{E}$ 称为差错序列, 或错误图样。 $\mathbf{E}$ 表示了 $\mathbf{R}$ 中具体那一位发生了错误, 即

$$e_i=0 \quad \text{表示第 } i \text{ 位无错} \quad r_i=c_i$$

$$e_i=1 \quad \text{表示第 } i \text{ 位有错} \quad r_i \neq c_i$$

把 $\mathbf{R}$ 代入 (10-3-7) 式中, 得

$$\mathbf{RH}^T = (\mathbf{C} \oplus \mathbf{E})\mathbf{H}^T = \mathbf{CH}^T \oplus \mathbf{EH}^T = \mathbf{EH}^T = \mathbf{S} \quad \text{或} \quad \mathbf{S}^T = \mathbf{HE}^T \quad (10-3-20)$$

其中, $\mathbf{S}=[s_{r-1}s_{r-2} \dots s_1s_0]$ 为 $1 \times r$ 阶行矢量。由式(10-3-20)可见 $\mathbf{S}$ 只与错误图样 $\mathbf{E}$ 有关, 而与发送的码字 $\mathbf{C}$ 无关。所以 $\mathbf{S}$ 称为监督矩阵为 $\mathbf{H}$ 的 (n, k) 线性分组码的伴随式。当 $\mathbf{E}=[000\mathbf{L} \ 00]_{1 \times n}$ 时, $\mathbf{S}=[000\mathbf{L} \ 00]_{1 \times r}$; 当 $\mathbf{E}$ 不为零时, $\mathbf{S}$ 不为零。译码器可通过伴随式 $\mathbf{S}$ 进行检错纠错。如果 $\mathbf{S}$ 为零, 则译码器判断接收码字正确, 并从该码字中除去监督位, 然后输出信息位。如果 $\mathbf{S}$ 不为零, 则必定有错, 由 $\mathbf{S}$ 可判断出错码的位置。下面举例说明。

对 $(7, 3)$ 码, 设 $\mathbf{C}=[1110100]$, 若发生一位错, 使 $\mathbf{R}=[11100*00]$ (* 表示错码), 则 $\mathbf{E}=[0000100]$, 可知

$$\mathbf{S}^T = \mathbf{H}\mathbf{E}^T = \begin{bmatrix} 1011000 \\ 1110100 \\ 1100010 \\ 0110001 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} s_3 \\ s_2 \\ s_1 \\ s_0 \end{bmatrix} \quad (10-3-21)$$

可见 $\mathbf{S}^T$ 刚好是错误图样 $\mathbf{E}$ 中 **1** 所对应的 $\mathbf{H}$ 中的一列。即 $\mathbf{R}$ 的第 i 位有错，则 $\mathbf{E}$ 的第 i 位为 **1**, $\mathbf{S}^T$ 与 $\mathbf{H}$ 中的第 i 列相同。判断出错误后, 可利用 $\mathbf{R} \oplus \mathbf{E} = \mathbf{C}$ 纠错。

对于前面所介绍的偶校验码, 当总码长为 n 时即为 $(n, n-1)$ 线性分组码。它只有一位监督码元 c_0 , 其构成的监督关系式见 (10-2-1) 式。在接收端进行解码校验时, 要判断接收到的码是否满足监督关系式 (10-2-1), 实际上就是计算

$$S = r_{n-1} \oplus r_{n-2} \oplus \dots \oplus r_1 \oplus r_0$$

当 $S = \mathbf{0}$ 时, 符合监督关系式, 判断接收到的码无错; 当 $S = \mathbf{1}$ 时, 不符合监督关系式, 就认为有错。 S 的取值只有两个, 它只能表示无错、有错两种状态, 而无法指出错在那一位。因此它只能检错不能纠错。如果再增加一位监督位, 相应的再增加一个监督关系式。那么 S 就有 4 种情况 **00**, **01**, **10**, **11**。用其中一种 **00** 表示无错, 剩余的 3 种能够用来指示一位错码的三种不同位置, 即具有纠错功能。同理, 如果有 r 个监督位, 即有 r 个监督关系式, 它可以指示出一位错码的 $(2^r - 1)$ 个可能的位置。

前述已知, 对 (n, k) 线性分组码, 有 $r = n - k$ 个监督关系式, 有 2^r 个不同的 S 。全 **0** 矢量表示无错, 所以 S 最多可指出 $2^r - 1$ 种错误。要纠正所有小于或等于 t 个错, 必须满足

$$2^r - 1 \geq C_n^1 + C_n^2 + \dots + C_n^t \quad (10-3-22)$$

式中， C_n^i 为组合数，也即

$$2^r \geq 1 + C_n^1 + C_n^2 + \dots + C_n^t = \sum_{i=0}^n C_n^i \quad (10-3-23)$$

式(10-3-23)说明了监督位数 r 与纠错能力的关系。当上式取等号时， 2^r 最小，即 r 达到满足要求时的最小值，此时监督位利用得最充分，称为完备码。

10.3.5 汉明码

汉明码是一种可以纠正单个随机错误的线性分组码。它是一种完备码，编码效率很高。在(10-3-22)式中，令 $t=1$ (即纠正一位错)，并取等号得

$$2^r - 1 = n \quad (10-3-24)$$

此时构成的 $(2^r - 1, 2^r - 1 - r)$ 线性分组码，称为汉明码。

汉明码具有以下特点 (m 为任意正整数， $m \geq 3$):

- (1) 监督位长 $r = m$ ，码长 $n = 2^r - 1 = 2^m - 1$ ；
- (2) 信息位长 $k = n - r = 2^r - 1 - r = 2^m - 1 - m$ ；
- (3) 最小码距 $d_0=3$ ，纠错能力 $t=1$ ；
- (4) 编码效率 $\eta = 1 - \frac{r}{n} = 1 - \frac{r}{2^r - 1}$ ， n 越大， η 越高。

下面给出 (7, 4) 汉明码的典型监督矩阵 H 和生成矩阵 G

$$H = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} = [P \mid I_3] \quad (10-3-25)$$

$$\mathbf{G} = \begin{bmatrix} 1000101 \\ 0100111 \\ 0010110 \\ 0001011 \end{bmatrix} = \left[\mathbf{I}_4 \mid \mathbf{P}^T \right] \quad (10-3-26)$$

汉明码只能纠正一位错，其最小码距 $d_0=3$ 。当码字中出现两位错时，它检测不出，故造成错判。如果在汉明码的基础上增加一位对所有码元都进行校验的监督码元，则监督码元的位数由原来的 m 变为 $m+1$ 。码长由原来的 2^m-1 变为 2^m ，信息位长度不变，形成 $(2^m, 2^m-1-m)$ 的线性码。称这种码为增余汉明码或扩展汉明码。增余汉明码的最小码距在汉明码的基础上增加了一位，变为 4。所以增余汉明码不仅能纠正一位错，同时也能检测 2 位错。

增余汉明码的构成是增加一位监督码元，使原汉明码的最小码重由奇数变为偶数。其监督矩阵可以由原汉明码的监督矩阵 $\mathbf{H}$ 得到

$$\mathbf{H}_e = \begin{bmatrix} 0 \\ 0 \\ \mathbf{H} \quad \mathbf{M} \\ 0 \\ 0 \\ \mathbf{111L} \quad 1 \end{bmatrix} \quad (10-3-27)$$

$\mathbf{H}_e$ 即为增余汉明码的监督矩阵，它在 $\mathbf{H}$ 的右边增加一列全 0，再在最后一行添加一行全 1 构成。

如把上述 (7, 4) 汉明码变为对应的 (8, 4) 增余汉明码，则其监督矩阵为

$$\mathbf{H} = \begin{bmatrix} 11101000 \\ 01110100 \\ 11010010 \\ 11111111 \end{bmatrix} \quad (10-3-28)$$

10.4 循环码

10.4.1 循环码的基本概念及码多项式

循环码是线性分组码中重要的一类，它是在严密的代数学理论基础上建立起来的。循环码的编码和解码设备通过反馈移位寄存器就可以实现，比较简单。其检错纠错能力也较强。在实际中得到了较大地发展。

循环码是一种线性分组码，它除了具有线性分组码的封闭性之外，还具有循环性。通常其前 k 位是信息码元，后 r 位为监督码元，具有系统码的形式。循环性是指：循环码组中的任一许用码字（全 0 码除外）循环左移（或循环右移）后所得到的码字仍为该循环码组中的一个许用码字。设码字矢量 $\mathbf{C}=[c_{n-1}c_{n-2}\text{L} \ c_1c_0]$ 是码长为 n 的循环码中的一个码字。对其进行循环左移、右移，无论循环左移、右移多少位，得到的结果均为该循环码中的一个码字。如下式中的各码字均为该循环码中的一个码字

$$\left. \begin{array}{l} c_{n-1}c_{n-2}\text{L} \ c_1c_0 \\ c_{n-2}c_{n-3}\text{L} \ c_0c_{n-1} \\ c_0c_{n-1}\text{L} \ c_2c_1 \end{array} \right\}$$

(10-4-1)

表 10-4-1 列出了一种（7，3）循环码的全部许用码字。

表 10-4-1 一种（7，3）循环码

序号	移位 次数	信息位	监 督 位	序号	移位 次数	信 息 位	监 督 位
0		0 0 0	0 0 0 0	4	6	1 0 0	1 1 1 0
1	0	0 0 1	1 1 0 1	5	4	1 0 1	0 0 1 1
2	5	0 1 0	0 1 1 1	6	3	1 1 0	1 0 0 1
3	1	0 1 1	1 0 1 0	7	2	1 1 1	0 1 0 0

为了便于用代数学的理论分析计算循环码，把循环码中的码字用多项式来表示，称为码多项式。它是把码字中各码元的取值作为码多项式的系

数。对于码字矢量 $\mathbf{C}=[c_{n-1}c_{n-2}\text{L} c_1c_0]$ 可以用码多项式表示为

$$T(x) = c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \text{L} + c_1x + c_0 \quad (10-4-2)$$

式中, x^i 是码元位置的标记, 它表示由其系数所决定的码元取值所处的对应位置。其系数只能取 **0** 或 **1**, 运算时其系数的运算为模 2 运算。如码字 **1001110**、**0011101** 用码多项式表示分别为

$$T_1(x) = x^6 + x^3 + x^2 + x \quad T_2(x) = x^4 + x^3 + x^2 + 1$$

且 $T_1(x) + T_2(x) = x^6 + x^3 + x + 1$ (即 **1001110** $\oplus$ **0011101** = **1010011**)

在整数的按模运算中, 最熟悉的是模 2 运算: $1+1 \equiv 0$ (模 2)、 $1+2 \equiv 1$ (模 2) 等。对于模 n 运算, 如果一个整数 m 可以表示为

$$\frac{m}{n} = Q + \frac{p}{n} \quad (p < n) \quad (10-4-3)$$

式中, Q 为整数, p 为 m 被 n 除后所得的余数。那么

$$m \equiv p \quad (\text{模 } n) \quad (10-4-4)$$

在多项式中同样可以进行类似的按模运算。如

$$\frac{F(x)}{N(x)} = Q(x) + \frac{r(x)}{N(x)} \quad (10-4-5)$$

式中, $N(x)$ 是幂次为 n 的多项式, $Q(x)$ 为商, $r(x)$ 为幂次低于 n 的余式, 多项式的系数在二元域上。式(10-4-5)可写为

$$F(x) = Q(x)N(x) + r(x) \quad (10-4-6)$$

所以

$$F(x) \equiv r(x) \quad (\text{模 } N(x)) \quad (10-4-7)$$

比如 x^3 被 (x^3+1) 除可得余式为 1, 则

$$x^3 \equiv 1 \quad (\text{模 } x^3+1) \quad (10-4-8)$$

同理有

$$x^4 + x^2 + 1 \equiv x^2 + x + 1 \quad (\text{模 } x^3 + 1) \quad (10-4-9)$$

循环码的码多项式符合如下定理。

定理 10.4.1 若 $T(x)$ 是长为 n 的循环码组中某个许用码字的码多项式，则 $x^i \cdot T(x)$ 在按模 $x^n + 1$ 运算下，也是该循环码组中一个许用码字的码多项式。

如(7,3)循环码中许用码字 **0011101** 的码多项式为 $T(x) = x^4 + x^3 + x^2 + 1$ ，则

$$\frac{x^3 T(x)}{x^7 + 1} = 1 + \frac{x^6 + x^5 + x^3 + 1}{x^7 + 1}$$

$$x^3 T(x) \equiv x^6 + x^5 + x^3 + 1 \quad (\text{模 } x^7 + 1)$$

$x^6 + x^5 + x^3 + 1$ 对应的码字为 **1101001**，它是该(7,3)循环码中的一个许用码字，而且它是上述循环码 **0011101** 左移 3 次后形成的。

证明： 设 $T(x) = c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \dots + c_2x^2 + c_1x + c_0$

那么

$$x^i \cdot T(x) = c_{n-1}x^{n-1+i} + c_{n-2}x^{n-2+i} + \dots + c_2x^{2+i} + c_1x^{1+i} + c_0x^i$$

$$\frac{x^i \cdot T(x)}{x^n + 1} = Q(x) + \frac{c_{n-1-i}x^{n-1} + c_{n-2-i}x^{n-2} + \dots + c_0x^i + c_{n-1}x^{i-1} + \dots + c_{n-i}}{x^n + 1}$$

$$= Q(x) + \frac{r_i(x)}{x^n + 1}$$

$$x^i \cdot T(x) \equiv r_i(x) \quad (\text{模 } x^n + 1) \quad (10-4-10)$$

其中， $r_i(x) = c_{n-1-i}x^{n-1} + c_{n-2-i}x^{n-2} + \dots + c_0x^i + c_{n-1}x^{i-1} + \dots + c_{n-i}$ 。它是 $T(x)$ 左移 i 位后形成的码字。若把 i 取不同的值重复做上述运算，可得到该循环码的其它许用码字。所以，码长为 n 的循环码的每一个许用码字都是按模 $x^n + 1$

运算的余式。如果已知码多项式 $T(x)$ ，则相应的循环码就可以由 $x^i \cdot T(x)$ 按模 $x^n + 1$ 运算的余式求得。

10.4.2 循环码的生成多项式及生成矩阵

从线性分组码的讨论中可以知道，有了生成矩阵就可以由信息位产生出相应的循环码。下面首先讨论有关生成多项式的定理。

定理 10.4.2 在循环码 (n, k) 中， $n-k$ 次幂的码多项式有一个，且仅有一个，用 $g(x)$ 表示。称这唯一的 $n-k$ 次多项式 $g(x)$ 为循环码的生成多项式。 $g(x)$ 的常数项不为零。

现在对上述定理加以说明。线性分组码的信息位和监督位之间成线性关系，可以由线性方程确定。所以当信息位为全 **0** 时，监督位必然也为全 **0**。循环码是一种线性分组码，它具有同样的性质。即信息位为全 **0** 时的循环码必然是全 **0** 码。在 (n, k) 循环码中，除全 **0** 码外，不可能出现连续 k 位均为 **0** 的码字。因为如果出现 k 个连续的 **0**，经过若干次移位后，将变为前 k 个信息码元全为 **0** 而监督码元不全为 **0** 的情况。这显然是不可能的。因此在 (n, k) 循环码的码集中，除全 **0** 码外，连续为 **0** 的长度最多只能有 $k-1$ 位。前 $k-1$ 位为 **0** 的码字，其末位必然为 **1**。因为，若其末位为 **0**，则经移位后必然出现 k 个连续的 **0**，这是不可能的。此前 $k-1$ 位为 **0**，末位为 **1** 的码字，所对应的码多项式是最高次幂为 $(n-1) - (k-1) = n-k$ 次的码多项式，而且它是循环码中幂次最低的码多项式。称它为循环码的生成多项式 $g(x)$ 。由该码字末位为 **1** 可知， $g(x)$ 的常数项不为零，这样的码多项式只有一个。因为如果有两个最高次幂为 $n-k$ 次的码多项式，则由循环码的封闭性可知，把这两个码字相加产生的码字连续前 k 位都为 **0**。这种情况不可能出现，所以在 (n, k) 循环码中，最高次幂为 $n-k$ 次的码多项式只有一个，生成多项式 $g(x)$ 具有唯一性。

一旦 $g(x)$ 确定, 则该 (n, k) 循环码就被确定了。 $g(x)$ 是循环码中幂次最低的码多项式。由它左移就可产生其它码多项式。比如 $xg(x)$ 、 $x^2g(x)$ 、 $x^3g(x)$ 等。用 k 个互相独立的码多项式 $g(x)$ 、 $xg(x)$ 、 $x^2g(x)$ 、...、 $x^{k-1}g(x)$ 可以构造出循环码的生成矩阵 $G(x)$ 为

$$G(x) = \begin{bmatrix} x^{k-1}g(x) \\ x^{k-2}g(x) \\ \vdots \\ x^2g(x) \\ xg(x) \\ g(x) \end{bmatrix} \quad (10-4-11)$$

例如, 有一个 $(7, 3)$ 循环码其最高次幂 $n-k$ 次的码字为 **0010111**, 其生成多项式 $g(x) = x^4 + x^2 + x + 1$ 。则利用(10-4-11)式可得其生成矩阵 $G(x)$ 为

$$G(x) = \begin{bmatrix} x^2g(x) \\ xg(x) \\ g(x) \end{bmatrix} \quad (10-4-12)$$

将此生成矩阵用系数表示, 写为生成矩阵 G , 则

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \quad (10-4-13)$$

式(10-4-13)不符合典型生成矩阵的形式, 所以它不是典型生成矩阵, 由它编出的码字不是系统码。但是对此矩阵作线性变化可以变换成典型生成矩阵的形式。

上例中设信息码为 $[c_6c_5c_4]$, 由生成矩阵多项式可以写出该循环码的码字为

$$\begin{aligned}
T(x) &= [c_6 c_5 c_4] G(x) = [c_6 c_5 c_4] \begin{bmatrix} x^2 g(x) \\ xg(x) \\ g(x) \end{bmatrix} \\
&= c_6 x^2 g(x) + c_5 xg(x) + c_4 g(x) \\
&= (c_6 x^2 + c_5 x + c_4) g(x) \\
&= u(x) g(x)
\end{aligned} \tag{10-4-14}$$

式中的 $u(x)$ 为信息码 $[c_6 c_5 c_4]$ 的多项式。

所以已知信息码 $U=[u_{k-1}u_{k-2} \dots u_1u_0]$ 和 $g(x)$ 后就可求得循环码的所有码多项式为

$$T(x) = [u_{k-1}u_{k-2} \dots u_1u_0] G(x) = u(x)g(x) \tag{10-4-15}$$

其中, $u(x)$ 为信息位所对应的多项式。信息位有 k 位, 所以 $u(x)$ 的最高阶数为 $(k-1)$ 次幂。

此种方法求得的码多项式为非系统码。从式(10-4-16)还可见, 所有的码多项式都可以被 $g(x)$ 整除。

定理 10.4.3 循环码 (n, k) 的生成多项式 $g(x)$ 是 x^n+1 的一个因式。

$g(x)$ 是最高次幂为 $n-k$ 次的码多项式。 $x^k g(x)$ 是最高次幂为 n 的多项式。利用定理 10.4.1 对 $x^k g(x)$ 作模 x^n+1 运算, 得

$$\frac{x^k g(x)}{x^n+1} = 1 + \frac{r(x)}{x^n+1} \tag{10-4-16}$$

式中, $r(x)$ 也是该循环码的一个码多项式, 它可以被 $g(x)$ 整除, 即 $r(x)=I(x)g(x)$ 。故式(10-4-16)可写为

$$x^k g(x) = (x^n+1) + r(x) = (x^n+1) + I(x)g(x)$$

对上式移项得

$$x^n+1 = x^k g(x) + I(x)g(x) = [x^k + I(x)]g(x) = h(x)g(x) \tag{10-4-17}$$

式(10-4-17)中, $h(x) = (x^n+1)/g(x)$ 为循环码的一致校验多项式, 可见 $g(x)$

是 x^n+1 的一个因式。利用这一特点可以产生 $g(x)$ 。其方法是对 (x^n+1) 进行因式分解，从中找出一个最高次幂为 $(n-k)$ 次且常数项不为零的因式，作为生成多项式 $g(x)$ 。

例如对于 $(7, 3)$ 循环码， $g(x)$ 的最高次幂为 4。可以从 (x^7+1) 中分解得到 $g(x)$ 。

$$x^7+1=(x+1)(x^3+x^2+1)(x^3+x+1) \quad (10-4-18)$$

生成多项式可选为

$$g_1(x)=(x+1)(x^3+x^2+1)=x^4+x^2+x+1 \quad (10-4-19)$$

或者

$$g_2(x)=(x+1)(x^3+x+1)=x^4+x^3+x^2+1 \quad (10-4-20)$$

两种生成多项式分别产生两种 $(7, 3)$ 循环码。

10.4.3 循环码的编码及解码

1. 循环码的编码

对 (n, k) 循环码，通过对 (x^n+1) 进行因式分解选择出生成多项式 $g(x)$ ，就可由信息码编出相应的循环码字。如何从 $g(x)$ 和信息码直接编出相应的系统码是下面要讨论的。设信息码多项式为 $m(x)$

$$m(x)=m_{k-1}x^{k-1}+m_{k-2}x^{k-2}+\dots+m_1x+m_0 \quad (10-4-21)$$

信息码多项式 $m(x)$ 的最高次幂为 $k-1$ 。将 $m(x)$ 左移 $n-k$ 位成为 $x^{n-k}m(x)$ ，其最高次幂为 $n-1$ 。 $x^{n-k}m(x)$ 的前一部分为连续 k 位信息码，后一部分为 $r=n-k$ 位的 **0**， r 正好是监督码的位数。所以在它的后一部分添上监督码，就编出了相应的系统码。监督码由监督关系确定，循环码的生成多项式 $g(x)$ 确定循环码，因此 $g(x)$ 也确定监督关系。从前述知，循环码的任何码多项式都可以被 $g(x)$ 整除，即 $T(x)=I(x)g(x)$ 。用 $x^{n-k}m(x)$ 除以 $g(x)$ ，得

$$\frac{x^{n-k}m(x)}{g(x)} = q(x) + \frac{r(x)}{g(x)} \quad (10-4-22)$$

所得的余式 $r(x)$ 的最高次幂为 $n-k-1$ 次, 即 $r(x) = r_{n-k-1}x^{n-k-1} + r_{n-k-2}x^{n-k-2} + \dots + r_1x + r_0$ 。将 $r(x)$ 作为监督位的多项式, 与 $x^{n-k}m(x)$ 模 2 相加, 形成新的多项式

$$T(x) = x^{n-k}m(x) + r(x) \quad (10-4-23)$$

由式(10-4-22)可见

$$x^{n-k}m(x) + r(x) = g(x)q(x) \quad (10-4-24)$$

所以 $T(x)$ 能被 $g(x)$ 整除, 其最高次幂为 $n-1$ 。 $T(x)$ 的前一部分为连续 k 位信息码, 后一部分为 $r = n-k$ 位的监督码。故 $T(x)$ 为循环码的码多项式, 而且是系统码。

对于这种编码方式, 具体可分为以下三步。下面通过一个 (7, 3) 循环码的例子具体说明。对 (7, 3) 循环码选择生成多项式为式(10-4-20), 即 $g(x) = x^4 + x^3 + x^2 + 1$, 并设已知信息码为 **111**。

(1) 将信息位 $m(x)$ 左移 $n-k$ 位成为 $x^{n-k}m(x)$ 。对上面的例子来说, 信息码 **111** 左移 4 位成为 **1110000**。

(2) 利用(10-4-22)式做除法, 求出余式 $r(x)$ 。对该例子来说, 为

$$\frac{x^6 + x^5 + x^4}{x^4 + x^3 + x^2 + 1} = x^2 + \frac{x^2}{x^4 + x^3 + x^2 + 1}$$

得余式为 $r(x) = x^2$, 即监督码的多项式。把此码多项式的形式用其系数代替, 写成码字的形式为

$$\frac{\mathbf{1110000}}{\mathbf{11101}} = \mathbf{100} + \frac{\mathbf{0100}}{\mathbf{11101}}$$

余式的码字为 **0100**。

(3) 构成系统码 $T(x) = x^{n-k}m(x) + r(x)$ 。对具体的例子为 **1110000 + 0100 =**

1110100。

上述编码过程可以用模 2 除法电路完成。它可由移位寄存器和模 2 加法电路实现。对上例中的 (7, 3) 循环码, $g(x) = x^4 + x^3 + x^2 + 1$ 时的编码器如图 10-4-1 所示。

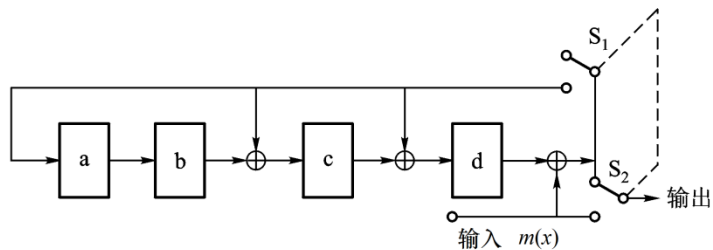


图 10-4-1 (7, 3) 循环码的编码器

编码器的工作过程如下：编码器工作之前先清零，使寄存器的初态为零，并使转换开关 S_1 、 S_2 均向下， S_1 使反馈线接通， S_2 使输入直接加到输出。然后开始输入三位信息码。输入的信息码一路直接送到输出端，作为系统码的前面一部分（即信息码部分）。另一路送入除法器作为被除数。三位信息码送完之后，使开关 S_1 、 S_2 均向上， S_1 使反馈线断开， S_2 使输出与除法器的输出相连，输入端输入的为零。在此种开关状态下，输出的是除法器的输出，即监督位。经过此两个阶段，输出端得到前面为信息码后面为监督码的系统码。表 10-4-2 示出上述编码过程中各点的状态变化过程。

表 10-4-2 (7, 3) 循环码的编码过程 (信息位为 111)

输入 m	移位寄存器				反馈	输出
	a	b	c	d		
0	初态				0	0
1	余数	1	0	1	1	1
1		0	1	0	0	1
1		0	0	1	0	1
0		0	0	0	无反馈	0
0		0	0	0		1
0		0	0	0		0
0		0	0	0		0

通常对于一个 (n, k) 循环码，若生成多项式为

$$g(x) = x^{n-k} + g_{n-k-1}x^{n-k-1} + \dots + g_1x + 1 \quad (10-4-25)$$

则对应的编码器如图 10-4-2 所示。

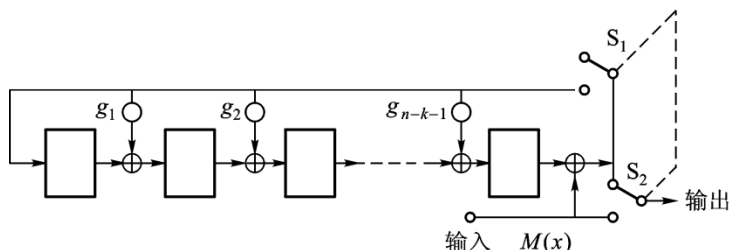


图 10-4-2 (n, k) 循环码的编码器

2. 循环码的解码

循环码的解码分检错和纠错两种情况。只进行检错的解码原理很简单。它是利用任何码多项式都可以被生成多项式 $g(x)$ 整除的原理实现的。设发送码字为 $T(x)$ ，接收到的码多项式为 $R(x)$ ，做除法有

$$\frac{R(x)}{g(x)} = q'(x) + \frac{r'(x)}{g(x)} \quad (10-4-26)$$

其中 $r'(x)$ 为余式。若余式 $r'(x)$ 为零，接收码字 $R(x)$ 能被整除，则 $R(x)=T(x)$ ，判断无错码。若余式 $r'(x)$ 不为零，即接收码字 $R(x)$ 不能被整除，则 $R(x) \neq T(x)$ ，判断有错码。可以通过 ARQ 差错控制方式使发端重发，得到正确的码字。

若要纠正错误，需要知道错误图样 $E(x)$ ，以便纠正错误。原则上纠错解码可按以下步骤进行。

- (1) 用生成多项式 $g(x)$ 除接收码字 $R(x)=T(x)+E(x)$ ，得到余式 $r'(x)$ ；
- (2) 按余式 $r'(x)$ 用查表的方法或通过某种运算得到错误图样 $E(x)$ ；
- (3) 从 $R(x)$ 中减去 $E(x)$ ，得到纠错后的原发送码字 $T(x)$ 。

第一步与检错解码相同，用除法器等就可实现。第三步做减法也较简

单。第二步可能需要较复杂的设备，并且在计算余式和决定错误图样 $E(x)$ 时，需要把接收码字 $R(x)$ 暂时存储起来。

例如一种 $(7, 3)$ 循环码纠单个错的解码器如图 10-4-3 所示。它包含除法器、缓冲器、门电路、及输出前做模 2 运算的异或。接收到的码字 $R(x)$ 输入后分两路。一路送入缓冲器暂存，另一路送入除法器做除法。当码字全部进入除法器后，若 $R(x)$ 能被 $g(x)$ 整除，则除法器中移位寄存器的状态全为零，说明接收码字为许用码字，判断为无错，直接将缓冲器暂存的 $R(x)$ 输出。若 $R(x)$ 不能被 $g(x)$ 整除，除法器中的存数指出错误位置。经移位（码字全部进入除法器后，再移位则输入为零）与门输出 $e = \bar{a}\bar{b}cd$ 在相应的出错码位上为输出 1。该 1 与缓冲器输出的错码模 2 相加，纠正错误，使输出码字正确。另一方面反馈回除法器，使各级移位寄存器清零。

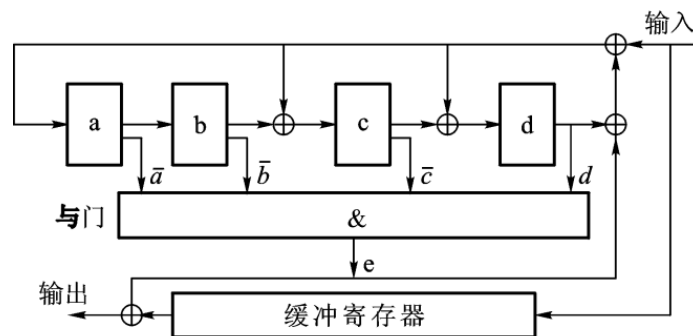


图 10-4-3 $(7, 3)$ 循环码纠单个错的解码器

实际中接收的码字是连续不断输入的，中间没有停顿。为了使解码器在移位纠错时不丢失输入的接收码字，需要两套除法电路及与门和一个缓冲寄存器配合使用。这种解码方法为捕错解码，又称梅吉特（Meggit）解码法。

循环码纠错能力强、编码解码设备简单，在实际中应用较广。实用的循环码种类很多。如费尔（Fire）码、高莱码(Golay)、BCH 码等。

10.4.4 BCH 码

BCH 码是取 Bose、Chandhari、Hocquenghem 三人姓名的开头字母 B、C、H 命名的一种循环码。

BCH 码是循环码的一个重要子类，它能纠正多个随机错误。BCH 码具有严密的代数结构，其生成多项式 $g(x)$ 和编码的最小距离 d_0 之间有密切的关系。可以根据需要构造出具有特定纠错能力的 BCH 码。BCH 码具有纠错能力强、构造方便、编码电路简单、译码也容易实现等特点，在实际中应用广泛。

最早的 BCH 码是定义在 $GF(2)$ 域¹上的二进制码，后来被推广到 $GF(q)$ 多元域上。BCH 码的参数为：码长 $n = 2^m - 1 (m > 3)$ ；校验位数 $n - k \leq mt$ ；最小距离 $d_0 \geq 2t + 1$ 。其中 m 和 t 为正整数。

在实际中为了较快地进行 BCH 码的设计选择，可以参考现成的 BCH 码生成多项式表。表 10-4-3 列出了部分本原 BCH 码的生成多项式，其中 n 为码长， k 为信息位长度， t 为所能纠错的位数，生成多项式 $g(x)$ 的系数用八进制表示，并从最高项开始。可以根据表 10-4-3 来选择所需 BCH 码。

表 10-4-3 部分本原 BCH 码生成多项式表

n	k	t	生成多项式 $g(x)$ ，用 8 进制表示
7	4	1	13
7	1	3	177
15	11	1	23
15	7	2	721
15	5	3	2467

¹ $GF(2)$ 域为二元域， $GF(q)$ 为多元域。要掌握 BCH 码的结构，需要了解有限域的知识，详细请见参考书[16]和[18]。

15	1	7	77777
31	26	1	45
31	21	2	3551
31	16	3	10765 7
31	11	5	54233 25
31	6	7	31336 5047
31	1	15	17777 77777 7
63	57	1	103
63	51	2	12471
63	45	3	17013 17
63	39	4	16662 3567
63	36	5	10335 00423
63	30	6	15746 41655 47
63	24	7	17323 26040 4441
63	18	10	13630 26512 35172 5
63	16	11	63311 41367 23545 3
63	10	13	47262 23055 27250 155
63	7	15	52310 45543 50327 1737
63	1	31	77777 77777 77777 77777 7

例如，要构造一个 $n=15$ 、 $t=3$ 的本原 BCH 码。查表 10-4-3 可得 $n=15$ 、 $t=3$ 时 $k=5$ ， $g(x)$ 的系数为八进制 2467。经变换 $(2467)_8 = (010\ 100\ 110\ 111)_2$ 。可得 $g(x) = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1$ 。得到了生成多项式 $g(x)$ ，即可用构造循环码的方法构造出本原 BCH 码。

10.4.5 CRC 码

循环冗余检验码（CRC, Cylic Redundancy Check）是一种简单有效的实用循环编码，其编码器和错误检测电路都很容易实现，已成为最常用的一种校验方式。因为 $r = n - k$ ，所以常将 (n, k) 循环码的编码称为CRC - r 编码。

实用的CRC编码已有多种生成多项式。表10-4-4列出的是已成为国际标准的CRC - r 编码。其中,CRC - 4在CCITT的G. 704建议中应用于2.048 Mb/s基群帧结构的校验,CRC - 12应用于字符长度为6比特的情况,CRC - 16、CRC - CCITT和CRC - 32应用于字符长度为8比特的情况。CRC - 12能够生成12个监督位。CRC - 16和CRC - CCITT生成16个监督位,CRC - 32生成32个监督位。即CRC - r 有 r 个监督位。前述已经介绍过，监督位的数目 r 越长，检错能力越强。所以在表10-4-4的CRC - r 编码中，CRC - 32的检错能力最强，它能检测长度不大于32位的突发错误，CRC - 32常用于局域网中。

表 10-4-4 部分CRC - r 编码

CRC - r 编码	生成多项式
CRC - 4	$g(x)=x^4+x+1$
CRC - 12	$g(x)=x^{12}+x^{11}+x^3+x^2+x+1$
CRC - 16	$g(x)= x^{16}+x^{15}+x^2+1$
CRC - CCITT	$g(x)= x^{16}+x^{12}+x^5+1$
CRC - 32	$g(x)= x^{32}+x^{26}+ x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

在本节前述介绍了循环码的编解码方式。确定了生成多项式即可进行编解码。在实际应用中，软件和硬件都可以实现CRC编码。用软件实现时，

其编解码速度会受到CPU速度的限制。用硬件方式实现时，其编解码速度有很大提高。由于CRC编解码非常简单，一般简单应用下都能达到速度要求。用硬件实现CRC有串行和并行两种方式。串行编码方式结构简单，能够工作在较高的时钟频率下，但编解码速度相对并行方式会慢一些。并行方式编码速度较快，但结构稍复杂。并行方式中的多级组合逻辑反馈电路会产生一定的延迟，在时钟周期很小、对时延要求很高时，要考虑电路的延迟是否超出了系统所能允许的延迟限度。

10.5 交织编码

10.5.1 交织编码的基本原理

前面讨论的主要是纠正随机错误的纠错编码。在实际的通信过程中，有时会出现突发干扰，它会在短时间内造成一连串的错码，一个码字内会出现多个码元的连续错误。这样由于错误码元太多，有时甚至成为另一个许用码字。这时用前面所讲的纠错编码方法就不能进行检错纠错。对抗这种突发错误，交织编码是一种有效的方法。交织编码能将聚集的突发错误分散化、不规则化，使其变换为随机误码。交织编码通过交织器实现，实际上是将原来的码流置乱，还原时再做解交织，还原为原来的顺序。交织在系统中的位置和作用如下：在发送端为“信道编码 - 交织 - 信道”，交织器把信道编码的输出序列按一定规律置乱，即重新排序。在接收端为“信道 - 解交织 - 信道解码”，解交织将序列还原为原来的顺序，即交织的逆过程。交织有分组交织、卷积交织，这里主要讨论分组交织。按照置乱的规律交织器可分为规则交织、不规则交织、伪随机交织等。规则交织即行列交织，将数据排成矩阵形式，按行输入按列输出，或者按列输入按行输出。不规则交织也是将数据排成矩阵形式，按行输入按对角线输出，或者其它顺序方式输出。伪随机交织是输入和输出之间的对应关系通过一串伪随

机码来对应。

下面通过一个简单例子讨论交织的原理。假设原来的码元序列为原序列 *A*，4 个码元为一个码字，见表 10-5-1。交织后的排序为序列 *B*，序列已经被置乱。传输后为序列 *C*，出现了突发错误，其中码元 3、7、11、15 发生了错误（有下划线表示出现错误），该码字的 4 个码元全都发生了错误，不能检纠错。解交织后的排序为序列 *D*，可见突发错误被分散，只要原来的编码能检纠一位错，即可实现检纠错。

表 10-5-1 交织原理的简单例子

原序列 <i>A</i>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	...
交织后 <i>B</i>	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16	...
传输后 <i>C</i>	1	5	9	13	2	6	10	14	<u>3</u>	<u>7</u>	<u>11</u>	<u>15</u>	4	8	12	16	...
解交织 <i>D</i>	1	2	<u>3</u>	4	5	6	<u>7</u>	8	9	10	<u>11</u>	12	13	14	<u>15</u>	16	...

10.5.2 交织编码的方法和性能

对于一个 (n, k) 线性分组码，可以通过交织构造一个 $(\lambda n, \lambda k)$ 线性分组码，称为交织码。其中 λ 称为交织深度或交织度。分组交织中原理最简明的是行列交织，它是通过建立矩阵，然后按照行进行编码和译码，如表 10-5-2 所示。

表 10-5-2 交织编码的构造和传输

	输出 列 1	输出 列 2	输出 列 3									输出 列 <i>n</i>
输入行 1	c11	c12	c13	...	...	...	...	...	...	...	...	c1 <i>n</i>
输入行 2	c21	c22	c23	...	...	...	...	...	...	...	...	c2 <i>n</i>
...	...	...	...	...	...	...	...	...	...	...	...	...

输入行 $\lambda-1$	...	...	...	...	...	...	...	...	...	...	...	...
输入行 λ	$c_{\lambda 1}$	$c_{\lambda 2}$	$c_{\lambda 3}$	...	...	...	...	...	...	...	...	$c_{\lambda n}$
	$(n-k)$ 位监督位					k 位信息位						

表 10-5-2 中每一行为该 (n, k) 线性分组码的一个码字，码长为 n 。共 λ 个码字构成 λ 行，形成一个 λ 行 n 列的矩阵，即 $(\lambda n, \lambda k)$ 的交织码。其编码的方法是把已编码的 (n, k) 码字按行输入随机存储器 (RAM)，再按列输出送至发送信道。这样就实现了交织过程。这一过程产生了对原始数据以 λ 个比特为周期的分隔效果，使传输中的突发错误在解交织后被分散了。在接收端的解交织操作与交织过程正好相反，即按列顺序写入 RAM，再按行读出，将数据还原为原来的顺序。然后再对每一行的一个码字进行纠错译码。可见如果在传输中出现突发错误，在收到的矩阵中它只可能按列的顺序出现，而纠错译码是按行进行的。解交织后突发错误被分散，使纠错成为可能。假设在传输中第 3 列整列出现突发错误，即突发错误长度为 λ ，解交织后每一行只有一位错。如果原来一行的码字能纠一位错，它就能纠该 λ 长的突发错误。假设在传输中第 2 列和第 3 列出现突发错误，即突发错误长度为 2λ ，解交织后每一行有 2 位错。如果原来一行的码字能纠 2 位错，它就能纠该 2λ 长的突发错误。长度为 $b\lambda$ 的突发错误被分散成每行中长度为 b 的错误，分别处于 λ 行中。如果原来的 (n, k) 线性分组码能纠 b 个错误，则交织码 $(\lambda n, \lambda k)$ 就能纠长度为 $b\lambda$ 的突发错误。当且仅当每行中的错误模式对原始码是可纠正的错误模式时，交织码矩阵的错误模式是可纠正的。交织码的码长扩大了 λ 倍，纠突发错误的能力也扩大了 λ 倍。但是它的编码效率并没有降低，仍和原来的 (n, k) 码相同。它所付出的代价是增加了存储设备 RAM 和加大了编解码延时。交织度 λ 越大，编解码延时越长，所需 RAM 越大。

10.6 卷积码

10.6.1 卷积码

1. 卷积码的概念

前面介绍的分组码是把信息序列分成长为 k 的许多子段，然后每个子段独立地编出各自的监督码形成长为 n 的码字。每个子段的监督位只与本子段的信息位有关，而与其它子段无关。各子段形成的码字在编码译码时各自独立进行。为了达到一定的纠错能力和效率，分组码的码字长度通常比较大。编码解码时必须把本子段的信息码存储起来再进行编码解码。这样当码长 n 较大时，编码解码过程所产生的时延也随之增加。若降低码长 n ，又会使纠错能力和效率下降。卷积码是由伊莱亚斯（Elias）于 1955 年首先提出的。它对信息位的处理与分组码完全不同，它是一种连续处理信息序列的编码方式。码字的监督位不仅和本段的信息位有关而且与其它段落的信息位也有关。整个编码过程前后相互关联，连续进行，所以又称为连环码。在编码时将信息序列分成长度为 k 的子段，把长度为 k 的信息比特编为 n 个比特， k 和 n 取值通常都很小，特别适合以串行形式传输信息，时延小。长为 n 的每个码字包括 k 个信息位和 $r = n - k$ 个监督位。但这里的监督位不仅与本段的 k 个信息位有关，也与前面 $(N - 1)$ 段的信息位有关， N 为相关联的信息序列的分段数目。编码后相互关联的码元数目为 nN 个。卷积码的纠错能力随 N 的增加而增大，差错率随 N 的增加而指数下降。目前还未找到严密的数学手段把卷积码的纠错性能与码的构成十分有规律地联系起来。通常采用计算机来搜索性能好的卷积码。在译码时，不但要从本段提取信息，还要提取出与此关联的前面 $(N - 1)$ 段的信息。与分组码相比，在设备复杂度相同的条件下卷积码的性能优于分组码，因此卷积码已广泛应用于实际通信系统中。

2. 卷积码的结构、原理

(1) 卷积码的结构、原理。

图 10-6-1 示出了卷积码编码器的一般结构。它由输入移位寄存器、模 2 加法器、输出移位寄存器三部分构成。输入移位寄存器共有 N 段，每段有 k 级，共 Nk 位寄存器，信息序列由此不断输入。输入端的信息序列进入这种结构的输入移位寄存器即被自动划分为 N 段，每段 k 位，它使输出的 n 个比特的卷积码与 N 段每段有 k 位的信息位相关联。通常把 N 称为约束长度（有些文献中把 $N-1$ 称为约束长度。由于该 N 段信息共有 Nk 个信息比特，所以也有称 Nk 为约束长度的）。一组模 2 加法器共 n 个，它实现卷积码的编码算法。输出移位寄存器，共有 n 级。输入移位寄存器每移入 k 位，它输出 n 个比特的编码。所以编码效率为

$$\eta = \frac{k}{n} \quad (10-6-1)$$

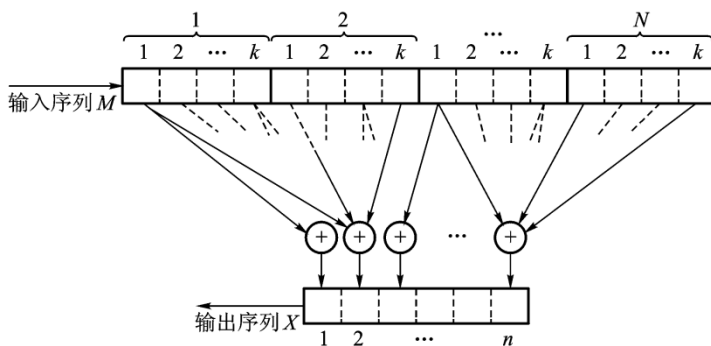


图 10-6-1 卷积码编码器的一般结构

此 n 比特的编码不仅与当前输入的 k 个信息位有关，而且与之前的 $(N-1)k$ 个信息位有关。对于具有上述结构的卷积码通常记作 (n, k, N) 。

图 10-6-2 示出一种 $(2, 1, 3)$ 卷积码编码器。其 $n=2$, $k=1$, $N=3$, 编码效率为 $\eta = \frac{1}{2}$ 。图中输出移位寄存器用开关代替。输入 $k=1$ 位信息比特，输出 $n=2$ 位编码。其输出方程为

$$x_{1j} = D_1 \oplus D_2 \oplus D_3, \quad x_{2j} = D_1 \oplus D_3 \quad (10-6-2)$$

其输入、输出对应关系如表 10-6-1 所示。

表 10-6-1 输入、输出对应关系

输入	m_1		m_2		m_3		m_4		m_5		
输出	x_{11}	x_{21}	x_{12}	x_{22}	x_{13}	x_{23}	x_{14}	x_{24}	x_{15}	x_{25}	

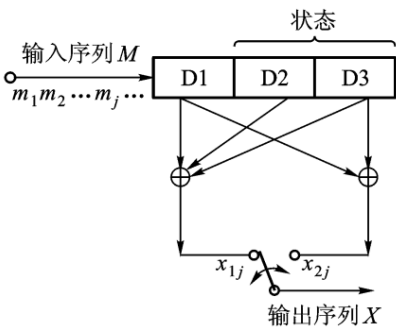


图 10-6-2 (2, 1, 3) 卷积码编码器

寄存器中新输入的信息码元进入 D_1 ，而以前存储的信息位进入 D_2D_3 ，称 D_3D_2 的状态为寄存器的状态（也即编码器的状态）。在本例中 D_3D_2 共有四种组合 **00、01、10、11**，所以寄存器共有四种状态。 D_3D_2 等于 **00、01、10、11**，分别对应状态 $a、b、c、d$ 。设寄存器的初态全为零，当第一个输入信息比特 $m_1=0$ 时，输出 $x_{11} \ x_{21} = 00$ ；若输入 $m_1=1$ ，则输出 $x_{11} \ x_{21} = 11$ 。当第二个信息比特 m_2 输入，第一个信息比特右移一位。第三个信息比特输入，第一、二个信息比特右移一位。此时两个输出由这三个信息比特决定。第四个信息比特移入时，第一个信息比特移出寄存器，不再起作用。在整个编码中每一位输出比特始终由存储于寄存器中的 3 位信息码元决定。其关系符合输出方程(10-6-2)。每个信息比特会影响到输出的 $nN=6$ 个码元。当输入信息序列为 **11010** 时，其移位编码过程如表 10-6-2 所示。

表 10-6-2 输入信息序列为 11010 时 (2, 1, 3) 卷积码编码过程

输入	寄存器状态	编码器	输出
----	-------	-----	----

符号	数值	D ₁ D ₂ D ₃	状态	符号	数值
		初态 000			
m_1	1	100	a	$x_{11} x_{21}$	11
m_2	1	110	b	$x_{12} x_{22}$	01
m_3	0	011	d	$x_{13} x_{23}$	01
m_4	1	101	c	$x_{14} x_{24}$	00
m_5	0	010	b	$x_{15} x_{25}$	10

(2) 生成多项式。

由移位寄存器和模 2 加法器构成的编码器可以用生成多项式表示。在图 10-6-2 所示的 (2, 1, 3) 卷积码编码器中, 针对输出 x_{1j} 、 x_{2j} , 编码器的生成多项式分别为

$$g_1(x)=1+x+x^2 \quad g_2(x)=1+x \quad (10-6-3)$$

设输入序列对应的多项式为 $M(x)$, 则 $x_1(x)=g_1(x)M(x)$ 、 $x_2(x)=g_2(x)M(x)$ 。

(3) 生成矩阵。

卷积码的输出与输入的关系也可以用生成矩阵描述。卷积码的输出序列可以看成是输入序列与编码器的冲激响应的卷积。当输入为单位冲激序列时(**100000**...), 上述(2, 1, 3)编码器输出冲激响应序列为 **1110110000**...。任一输入序列都可看成是单位冲激序列不同时延的线性组合, 因此其输出也为其冲激响应序列不同时延的线性组合。所以卷积码的生成矩阵可写为

$$G = \begin{bmatrix} \mathbf{1110110000L} \\ \mathbf{01110110000L} \\ \mathbf{001110110000L} \\ M \end{bmatrix} \quad (10-6-4)$$

3. 卷积码的图解表示

图解表示卷积码的方法有三种: 树状图 (又称码树图)、网格图、状态图。

(1) 树状图。

本例 (2, 1, 3) 卷积码的树状图如图 10-6-3 所示。

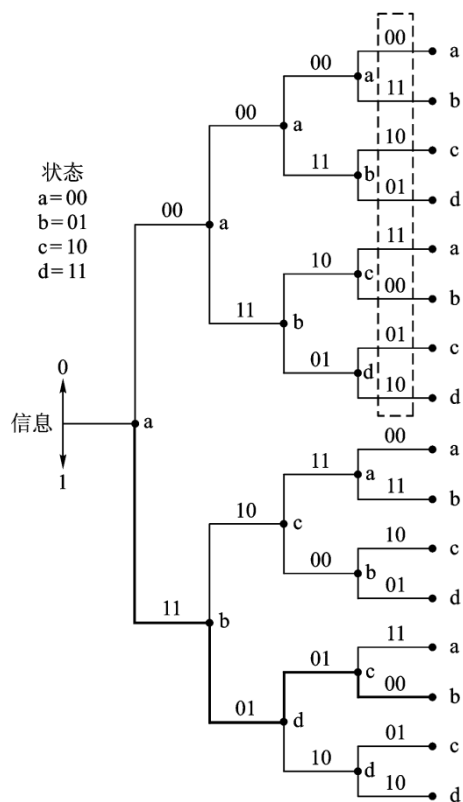


图 10-6-3 (2, 1, 3) 卷积码的树状图

图 10-6-3 中每一分支代表一个输入比特。一般输入为 **0** 对应上分支，输入为 **1** 对应下分支，每个分支上面标出与输入对应的输出码。设输入移位寄存器的初态全为 **0**，树状图从节点 a（即寄存器的状态为 $D_3 D_2=00$ ，对应状态 a）开始画起。当第一个输入信息位 $m_1=0$ 时，对应上分支输出为 $x_{11} x_{21}=00$ ，存储器状态不变仍为状态 a。若 $m_1=1$ ，对应下分支，输出为 $x_{11} x_{21}=11$ ，存储器状态变化， $D_3 D_2=01$ ，对应状态 b。共有两条支路。当输入第二个信息位 m_2 时，根据寄存器原状态不同和 m_2 的取值不同可分为四个支路。原状态为 a 时： $m_2=0$ ，从 a 出发画上分支， $D_1 D_2 D_3=000$ ，输出为 **00**； $m_2=1$ ，从 a 出发画下分支， $D_1 D_2 D_3=100$ ，输出为 **11**。原状态为 b 时： $m_2=0$ ，从 b 出发画上分支，输出为 **10**； $m_2=1$ ，从 b 出发画下分支，输出为 **01**。以此类推，可画出图 10-6-3 的二叉树图形。输入信息序列长度增加一位，树

状图分支增加一倍。对于第 j 个输入信息比特有 2^j 条支路。由于卷积码的约束长度为有限值 N ，当 $j \geq N$ (本例中 $N = 3$) 时，树状图的节点自上而下开始重复出现 2^{N-1} 种状态 (本例中为 4 种状态)，如图 10-6-3 中虚框所示。当输入的信息序列为 **11010** 时，在树状图中所对应的路径如图中粗线所示。相应的输出序列为 **1101010010**。

(2) 网格图。

树状图具有重复性，利用这一特点将其变化为另一种更为紧凑的表示形式，如图 10-6-4 所示，称为网格图。

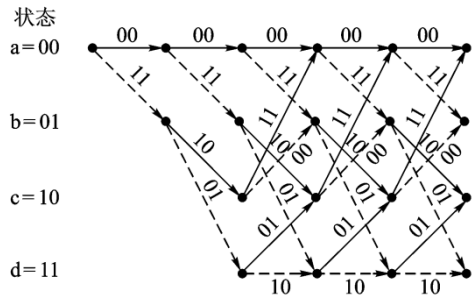


图 10-6-4 (2, 1, 3) 卷积码的网格图

由树状图得到网格图的方法是：把树状图中具有相同寄存器状态的节点合并在一起，画于同一行中。输入为 **0** 对应上分支，用实线表示。输入为 **1** 对应下分支，用虚线表示。各分支上标出对应的输出。四行节点即存储器的四种状态 a 、 b 、 c 、 d 。一般约束长度为 N ，则有 2^{N-1} 种状态。随着输入信息序列的增加，网格图的节向右延伸。从第 N 节开始，网格图的图形开始完全重复。和树状图一样，每种输入序列都对应着网格图中一条相应的路径。如输入序列 **11010** 对应的路径如图中粗线所示，其输出序列为 **1101010010**。

(3) 状态图。

由上述例子可见，移位寄存器的状态对应着编码器的状态 a 、 b 、 c 、 d 。编码器的输出由其输入和编码器的状态所决定。每一次输入都使移位寄存

器移位，编码器状态变为一新的状态。由此可画出编码器的状态转移图，如图 10-6-5(a)所示。由于网格图也表示了编码器的状态变化过程，所以该状态图也可由网格图得到，如图 10-6-5(b)所示。

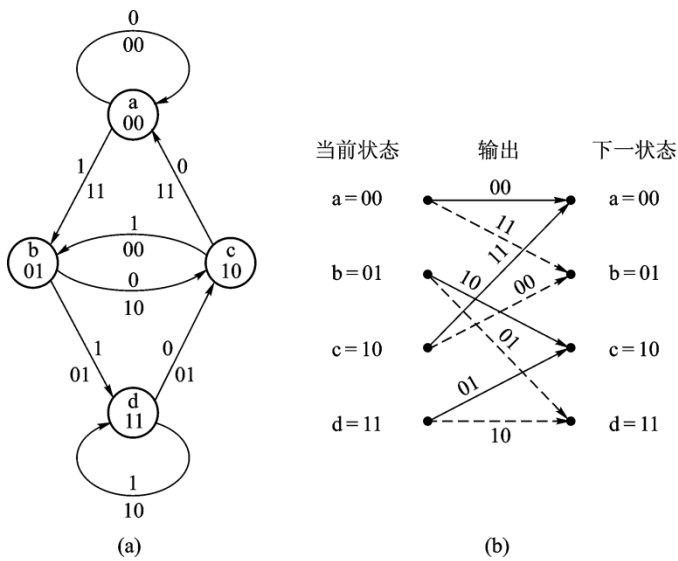


图 10-6-5 (2, 1, 3) 卷积码的状态图

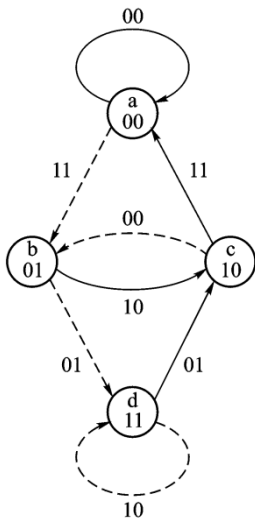


图 10-6-6 (2, 1, 3) 卷积码的状态图

图 10-6-5 (a) 中四个圆圈表示 (2, 1, 3) 卷积码编码器的四种状态。状态转移路线上面为输入比特，下面为相应的输出比特。由于输入比特只

可能取 **0**、**1** 两种，也可以用虚线表示输入比特为 **1**，用实线表示输入比特为 **0**，而在转移路线上只标出输出比特，如图 10-6-6 所示。

(4) 距离的定义。

对于一种编码其纠错能力的大小与码距密切相关。在线性分组码中常以最小距离来度量其纠错能力的大小。卷积码距离的定义有多种。根据所取的编码比特组的不同而不同。通常使用的有最小距离 $d_{\min}$ 和自由距离 d_{free} 。对于卷积码 (n, k, N) ，卷积码中长度为 nN 的编码后序列之间的最小汉明距离称为最小距离 $d_{\min}$ 。任意长编码后序列之间的最小汉明距离称为自由距离 d_{free} 。当译码算法仅限于处理长度为 nN 的接收序列时，最小距离 $d_{\min}$ 是一个重要参量。当译码时所考察的编码后序列长度大于 nN 时，自由距离 d_{free} 是一个重要参量。

4. 卷积码的译码

与分组码类似，卷积码也可用来检错和纠错。卷积码的译码可以分为代数译码和概率译码两大类。

(1) 代数译码。

代数译码是基于码的代数结构进行的，即信息码元和监督码元之间的约束关系、生成矩阵等，如门限译码。门限译码是一种较早使用的卷积码的译码方法。它设备简单，译码速度快，它主要用于系统卷积码的译码，适合于有突发错误的信道。门限译码的误码性能比目前应用的概率译码差。门限译码的原理以分组码为基础，它把卷积码看成是在译码约束长度含义下的分组码。通过计算伴随式等对接收到的码字进行译码纠错输出。

(2) 概率译码。

概率译码是根据信道统计特性，从概率的角度进行译码的。目前概率译码已成为卷积码主要的译码方式。主要有两种译码技术：维特比译码和序列译码。它们都是建立在最大似然译码基础之上的。图 10-6-7 示出了卷

积码的编译码系统模型。

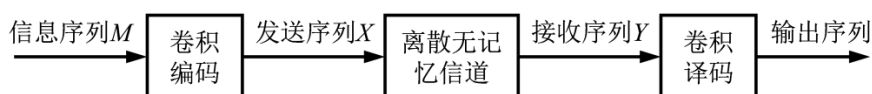


图 10-6-7 编译码系统模型

输入序列 M 经卷积码编码成为发送序列 X ，此 X 序列可用树状图或网格图中的某一特定路径来表示。发送序列 X 经过有噪声的离散无记忆信道后接收端收到序列 Y 。受到信道噪声的影响，接收序列有可能不同于发送序列。离散无记忆信道是一种数字输入和数字输出的（或称为量化输出）信道模型。其输入 X 是一个二进制符号序列，输出 Y 是具有 J 种符号的序列。如 X 序列发出一个符号 $x_i (i = 0, 1)$ ，信道输出端收到一个相应的符号 $y_j (j = 0, 1, 2, \dots, J-1)$ 。由于信道无记忆 y_j 只与 x_i 有关。若 $J = 2$ ，则输出是一个二进制序列。如果各种概率满足下列关系： $P(y_0 | x_0) = P(0|0) = P(y_1 | x_1) = P(1|1)$ ， $P(y_0 | x_1) = P(0|1) = P(y_1 | x_0) = P(1|0)$ ，称该信道为二进制对称信道，或硬判决信道。若 $J > 2$ ，称为软判决信道。所以概率译码又分为硬判决和软判决两种。复杂的译码器就是采用软判决信道，其性能较好。

概率译码考察输出序列，以判定长度为 L 的 2^L 个可能发送的序列中究竟是哪一个进入了编码器。最大似然译码是将接收序列与所有可能的传输序列进行比较，从中选择对应最大似然函数的传输序列，然后确定译码器的输出。发送序列 X 变成接收序列 Y 的条件概率 $P(Y|X)$ 称为似然函数， $\log P(Y|X)$ 为对数似然函数。在收端只知道接收序列 Y ，译码的任务就是要寻找具有最大似然函数的发送序列 X 。对于二进制对称信道，设 X 序列的长度为 L 个符号， $P(0|1) = P(1|0) = P$ ，在传输中产生了 e 个符号的错误，即 Y 与 X 有 e 个位置上符号不同， $d(Y, X) = e$ ，则

$$\log P(Y|X) = \log[P^e(1-P)^{L-e}] = L \log(1-P) - e \log \frac{1-P}{P}$$

对于 $P < \frac{1}{2}$, $\log \frac{1-P}{P} > 0$ 。因此汉明距离 $d(Y, X) = e$ 最小等同于对数似然函数最大。即对于二进制对称信道, 最大似然译码等于最小汉明距离译码。

即在汉明距离 $d(Y, X)$ 最小时, $\log P(Y|X)$ 最大。寻找 $\log P(Y|X)$ 最大就相当于寻找最小汉明距离 $d(Y, X)$ 。卷积码的发送序列 X 都对应着编码器树状图或网格图中的一条路径。在进行解码算法时也是对可能的路径计算其最小汉明距离 $d(Y, X)$ (或似然函数) 并进行比较选择。在卷积码的译码中, 通常称可能的译码序列与接收序列之间的汉明距离为量度。对于长为 L 的二进制序列, 要对 2^L 条路径的汉明距离 (或似然函数) 进行计算比较, 选取其中汉明距离最小 (或似然函数最大) 的一条作为最佳路径。译码的计算量随 L 的增加指数增长。

维特比译码是建立在最大似然译码基础之上的。它利用了网格图的重复性结构, 降低了所需的计算次数。它延伸所有状态下可能的最佳路径, 然后比较选择。有关维比特译码的过程及算法分析与实现请参考有关文献。

序列译码也是建立在最大似然译码基础之上的, 它以路径的度量为准, 选择出与接收序列最相近的路径。序列译码中, 一次只搜索一条可能路径, 其计算量和存储量大大减少。但在这种只延伸一条路径的有限搜索情况下, 如果后来发现此条路径不符合要求, 则要退回重新搜索。序列译码有两种主要算法: 费诺算法和堆栈算法。有关序列译码的详细内容请参考有关文献。

10.6.2 网格编码调制 (TCM)

调制解调技术和差错控制编码技术是数字通信系统的两个基本问题。在前面的讨论中, 这两个问题是分别独立考虑的。它们对误码性能的改善是以系统传输带宽的增加换取的。在带限信道中, 希望在不增加传输带宽

的条件下，使差错率降低。网格编码调制（TCM）可以达到这一目的。它把编码与调制综合考虑统一设计。

通常用汉明距离来说明分组码或卷积码的抗干扰性能。而在网格编码调制中，影响系统差错率的重要参数是信息点之间的欧几里德距离（简称欧氏距离）。所以网格编码调制就是要想法增加这个距离，以改善系统的误码性能。纠错编码通常是以汉明距离作为度量标准的。但当把编码和调制作为一个整体考虑时，对于具有最大汉明距离的卷积码，已调信号不一定具有最大欧氏距离。对于 2PSK 调制，汉明距离与欧氏距离等价，按照汉明距离最佳而设计的卷积码对信号的欧氏距离来说也是最佳的。而在一般的多进制调制中，汉明距离与欧氏距离不等价。因此要把编码与调制综合考虑统一设计，使编码器和调制器级联后产生的编码信号序列具有最大的欧氏距离。

实现 TCM 的关键是确定编码符号到调制信号的映射规则。通常可采用集合划分映射。图 10-6-8 画出了 8PSK 信号空间划分成子集的情况。所有八个信号点分布在一个圆周上，最小欧氏距离 $\Delta_0 = 2 \sin \frac{\pi}{8} = 0.765$ 。第一次划分将八个信号点划分为两个子集，每个子集各含有四个信号点，同一子集中信号点之间的最小欧氏距离为 $\Delta_1 = \sqrt{2}$ 。由图 10-6-8 可见，经过三次划分后，分别产生 2、4、8 个子集。它们具有以下特点：两个信号点之间的最小欧氏距离逐次增大，即 $\Delta_0 < \Delta_1 < \Delta_2$ 。所以，集合划分映射是把信号点不断的分解为 2、4、8、……个子集，使它们中信号点之间的最小欧氏距离不断增大。图 10-6-9 示出编码的 16QAM 方式的集合划分过程。可见子集最小欧氏距离也是逐渐增大的。即 $\Delta_0 < \Delta_1 < \Delta_2 < \Delta_3$ 。

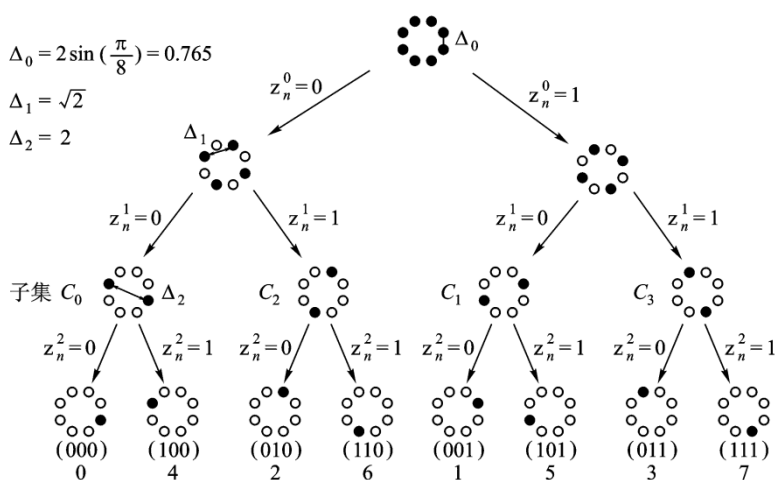


图 10-6-8 8PSK 信号集合划分图

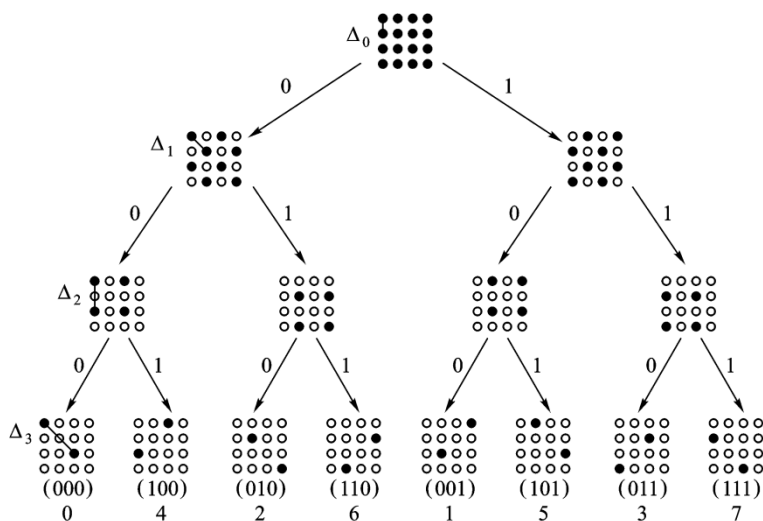
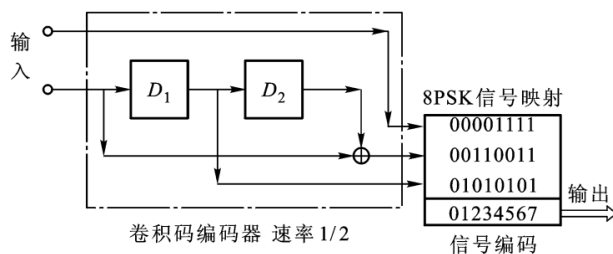
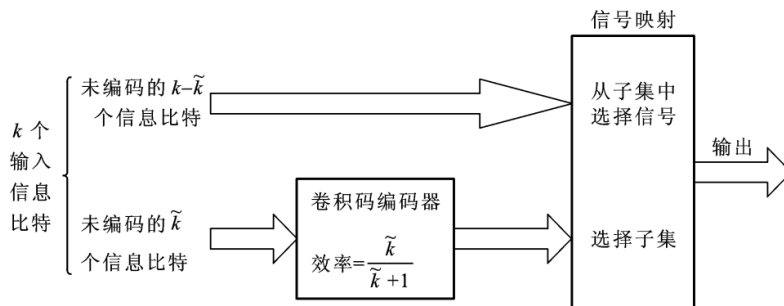


图 10-6-9 16QAM 方式的集合划分

根据上述集合划分映射的原理，可以得到 TCM 编码器的一般结构如图 10-6-10 所示，它由卷积码编码器和信号映射两部分构成。TCM 信号是通过如下方式产生的：在每一编码调制间隔，有 k 比特待传输的信息送入。此 k 比特未编码的输入信息分为两部分，一部分经卷积码编码器，编码输出用于选择相应的子集，另一部分不经过编码用于确定信号与子集中信号点之间的映射关系。其中的 k' 比特 ($k' < k$) 通过一速率为 $\frac{k'}{k+1}$ 的二进制卷积码

编码器，输出 $k-k'$ 编码比特。调制信号集合分成 $2^{k'}$ 个信号子集，每个子集中含 $2^{k-k'}$ 个信号。卷积码编码器输出的 $k-k'$ 编码比特用来选择 $2^{k'}$ 个信号子集中的一个。剩余的 $k-k'$ 个未编码输入信息比特用于选择传送该子集的 $2^{k-k'}$ 个信号中的某一个。



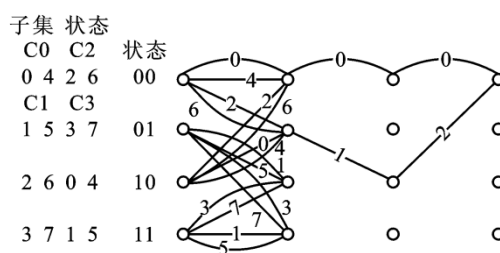


图 10-6-12 四状态 8PSK TCM 编码器的网格图

10.6.3 Turbo 码

香农有扰信道的编码定理指出：在有扰信道中只要信息的传输速率 R 小于信道容量 C ，总可以找一种编码方法，使信息以任意小的差错概率通过信道传送到接收端，即误码率 P_e 可以任意小，而且传输速率 R 可以接近信道容量 C 。香农有扰信道的编码定理本身并未给出具体的纠错编码方法，但它为信道编码奠定了理论基础，从理论上指出了信道编码的发展方向。很多科技工作人员为此进行了不断地探索，产生了许多有效的信道编码方法，纠错码的性能也越来越好。但从实际应用来看，各种纠错码的性能与香农在信道编码定理中给出的极限仍有差距。1993 年法国科学家 C. Berrou 等人发表了一篇论文“接近香农极限的纠错编码和译码：Turbo 码”。Turbo 码一经出现，立即引起了全世界信道编码学术界的广泛关注。Turbo 码是近年来纠错编码领域研究的重要突破。Turbo 码是一种并行级联码，它的内码，外码均使用卷积码。它采用了迭代译码方法，挖掘了级联码的潜力。计算机仿真模拟的结果表明，在加性高斯白噪声无记忆信道（AWGN）上，特定参数条件下 Turbo 码的性能可以达到与香农理论极限相差 0.7dB 的性能，接近香农极限。Turbo 码的优异性能吸引了许多科技工作人员对此进行研究，使 Turbo 码有了很大的发展，并且在多方面得到了实际应用。Turbo 码已经被确定为第三代移动通信系统 IMT-2000 中高质量、高速率传输业务的首选编码方案。WCDMA、cdma2000 和我国的 TD-SCDMA 的信道编码方案都

使用了 Turbo 码。

1. Turbo 码编码器

Turbo 码编码器的典型结构如图 10-6-13 所示。它由分量编码器、交织器、删余及复用三部分组成。

输入信息序列 M ，一路直接送到复用器，作为信息比特；另一路送入分量编码器 1 进行编码，编码后的输出 X_{p1} 送入删余器，经删余后得到校验码 X'_{p1} ；还有一路送至交织器，信息序列 M 经过交织器后再送入分量编码器 2，分量编码器 2 的输出 X_{p2} 经删余后得到另一校验码 X'_{p2} 。最后信息比特和校验码复用后形成 Turbo 码序列 X 。

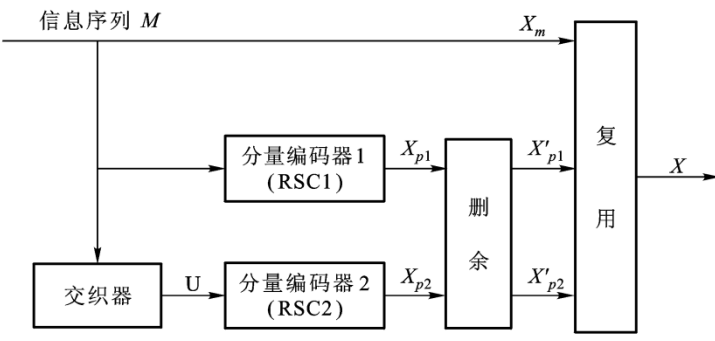


图 10-6-13 Turbo 码编码器

(1) 分量编码器

研究实践表明，递归系统卷积码 RSC (Recursive Systematic Convolutional) 比非递归的非系统卷积码 NSC (Non Systematic Convolutional) 具有更好的性能。目前，典型的 Turbo 码编码器采用反馈型递归系统卷积码 RSC。即分量编码器 1 和分量编码器 2 分别为 RSC_1 和 RSC_2 。 RSC_1 和 RSC_2 可以相同也可以不同。

递归系统卷积码 RSC 可以由非系统卷积码 NSC 转化后得到。图 10-6-14 为一个非系统卷积编码器，图 (a) 为 NSC 编码器框图，图 (b) 为 NSC

编码器电路图。其生成多项式分别为

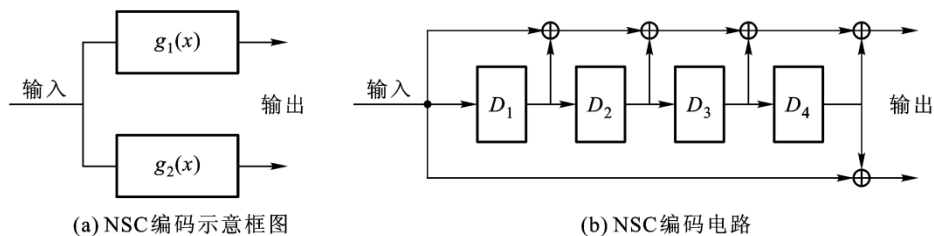


图 10-6-14 (37, 21) 非系统卷积编码器

$$g_1(x) = x^4 + x^3 + x^2 + x + 1 \quad (10-6-5)$$

$$g_2(x) = x^4 + 1 \quad (10-6-6)$$

用矢量表示为: $g_1 = [011111]$ $g_2 = [010001]$

写成八进制形式即 (37, 21)。也可以写成生成矩阵形式, 为

$$G(x) = \begin{bmatrix} x^4 + x^3 + x^2 + x + 1 & x^4 + 1 \end{bmatrix} \quad (10-6-7)$$

对该矩阵的第一行各项除 $x^4 + x^3 + x^2 + x + 1$, 可得

$$G'(x) = \begin{bmatrix} 1 & \frac{x^4 + 1}{x^4 + x^3 + x^2 + x + 1} \end{bmatrix} \quad (10-6-8)$$

$$g'_1(x) = 1 \quad g'_2(x) = \frac{x^4 + 1}{x^4 + x^3 + x^2 + x + 1}$$

画出此新的编码器的框图如图 10-6-15 (a) 所示。其对应的编码器电路图

如图 10-6-15 (b) 所示。它是一个递归系统卷积码 RSC。

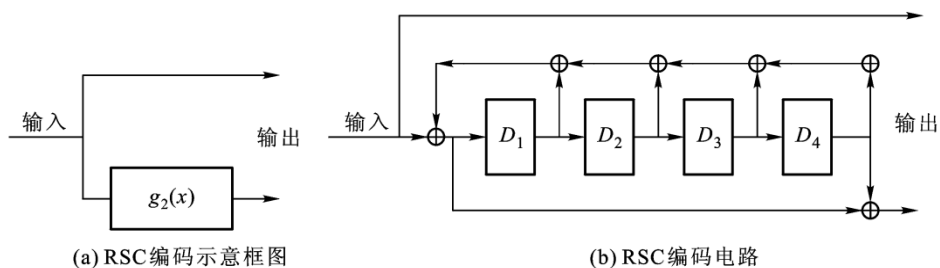


图 10-6-15 递归系统卷积码 RSC

(2) 交织器

交织器是 Turbo 码的关键部件之一,它对 Turbo 码性能的影响非常重要。在一般传统的信道传输时,交织器使突发产生的集中错误分散化,其目的是抗信道突发错误。在 Turbo 码中,运用交织器是置乱原始数据的排列顺序,改变码的重量分布,提高输出 Turbo 码的整体性能。

在 Turbo 码中常用的为随机交织器。随机交织器其交织过程的映射规律是随机的,即数据写入存储器和从存储器中读出的地址对应是随机的。但完全的随机交织在接收端很难进行解交织以便恢复原来的信息。或者说要把每一次交织过程的映射规律传送到接收端,接收端才能进行解交织,其传输的工作量太大,它会增加信道负担和译码器复杂度,不实用。实际中使用的为伪随机交织器。它的关键是要选取一定的伪随机序列,由伪随机序列确定交织过程的映射规律。

(3) 删余及复用

删余与复用的目的是为了得到合适的码率。从图 10-6-13 可见, Turbo 码编码器中有两个分量编码器 RSC_1 和 RSC_2 。两个 RSC 分量编码器不一定完全相同,设其码率分别为 R_1 和 R_2 。如果不做删余及复用处理,合成后的码率 R 与 R_1 、 R_2 的关系为

$$\frac{1}{R} = \frac{1}{R_1} + \frac{1}{R_2} - 1$$

两个分量编码器 RSC_1 和 RSC_2 产生两个序列 X_{p1} 、 X_{p2} ,如果不进行删余处理,则在输出码流中的冗余比特太多。为了提高码率,使序列 X_{p1} 、 X_{p2} 经过删余器,按照一定规律删除一些校验比特,形成新的校验序列 X'_{p1} 、 X'_{p2} 。比如在图 10-6-13 中两个分量编码器 RSC_1 和 RSC_2 的码率为 $\frac{1}{2}$,如果不做

删余处理直接复用,则得到 Turbo 码的码率为 $\frac{1}{3}$ 。为了产生码率为 $\frac{1}{2}$ 的 Turbo 码,可以从 RSC₁ 和 RSC₂ 的输出分别删去 1 比特,即校验序列在 RSC₁ 和 RSC₂ 的输出间轮流取值,经过复用就得到了码率为 $\frac{1}{2}$ 的 Turbo 码。

Turbo 码编码器也可以由二维扩展到多维,如图 10-6-16 所示。

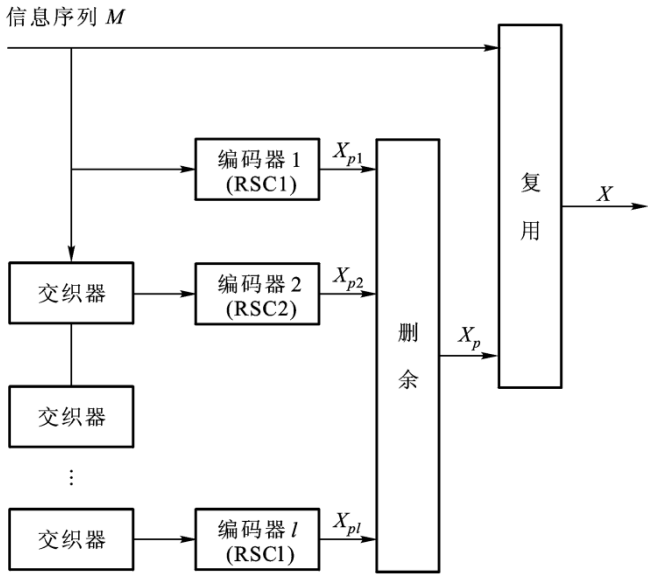


图 10-6-16 Turbo 码编码器的一般结构

2. Turbo 码译码器

Turbo 码的译码通常是运用最大似然译码准则,采用迭代译码的方法实现的。图 10-6-13 所示的 Turbo 码编码器含有两个分量编码器,与此对应的 Turbo 码译码器也有两个分量译码器 DEC₁、DEC₂。Turbo 码译码器的典型结构如图 10-6-17 所示。它由两个分量译码器及相应的交织器和解交织器组成。其中的分量译码器均采用软输入、软输出(SISO, Soft Input Soft Output)译码器。分接与内插是对接收序列进行处理,其功能与编码器中的删余及复用刚好相反。接收到的数据流结构是经删余及复用后的数据流:信息码加校验码。它要经过解复用,对数据流分接与内插后,恢复成删余及复用前 X_m 和 X_{p1} 、 X_{p2} 的结构,然后分别送入相应的分量译码器。DEC₁ 对 RSC₁

进行最佳译码，DEC₂对 RSC₂进行最佳译码。由于两个分量来自于同一个输入信息序列 M ，必然具有一定的相关性，可以互为参考。所以在 Turbo 码译码器中，将 DEC₁的软输出经交织器后作为附加信息送入 DEC₂，使输入到 DEC₂的原始信息增加，提高译码的正确性。同样，将 DEC₂的软输出经解交织后作为附加信息送入 DEC₁。经过多次迭代得到对应于输入信息序列 M 的最佳值 $\hat{M}$ 作为译码输出。

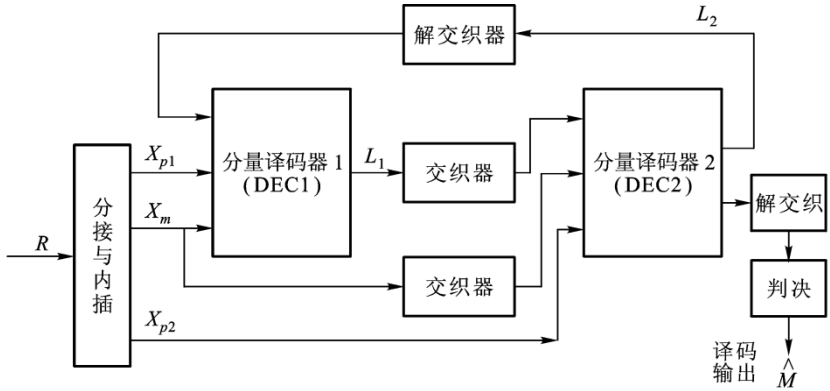


图 10-6-17 Turbo 码译码器

在 Turbo 码译码器中采用的软输入、软输出迭代译码算法有多种。常见的如标准 MAP 算法 (MAP, Maximum A posteriori Probability, 最大后验概率算法)、对数 MAP 算法 (log-MAP 算法)、max-log-MAP 算法、软输出维特比译码 (SOVA, Soft Output Viterbi Algorithms) 等。对具体的算法请参看有关文献，本章不做详细介绍。

习题

- 10-1 请说明随机信道、突发信道、混合信道各自的特点。
- 10-2 请说明差错控制方式的目的是什么？常用的差错控制方式有哪些？
- 10-3 请说明 ARQ 方式有哪几种？

10-4 已知线性分组码的八个码字为：**000000, 001110, 010101, 011011, 100011, 101101, 110110, 111000**，求该码组的最小码距。

10-5 上题给出的码组若用于检错，能检出几位错码？若用于纠错，能纠几位错？若同时用于纠错、检错如何？

10-6 若两个重复码字 **0000, 1111**，纠检错能力如何？

10-7 写出 $k = 1, n = 5$ 时重复码的一致检验矩阵 **H** 及生成矩阵 **G**，并讨论它的纠、检错能力。

10-8 现考虑一简单的把每一信息符号重复 5 次生成一个码字的纠错编码系统。

- ① 求编码效率；
- ② 编码中有多少个码字？
- ③ 最小码间距离是多少？
- ④ 发生多少个信道错误就会使得其中一个码字混淆为另一个码字？
- ⑤ 编码中所用到的码字占所有可能码字的比例是多少？

10-9 某 BSC 信道的错误概率为 $p = 0.01$ ，在其上传送码字 $x = \mathbf{0110100}$ ，接收到的码字为 $y = \mathbf{0010101}$ 。问发生这一事件的概率为多少？求引起这一错误的错误矢量。

10-10 写出 $n = 7$ 时偶校验码的一致校验矩阵 **H** 和生成矩阵 **G**，并讨论其纠、检错能力。

10-11 一个线性分组码的校验矩阵

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

试求该码的生成矩阵与最小码间距离。

10-12 一线性编码的监督矩阵为

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

问下列矢量是否为编码中的码字？

$$y_1 = 0011000$$

$$y_2 = 1010001$$

$$y_3 = 1110100$$

$$y_4 = 0101011$$

$$y_5 = 1101000$$

10-13 令 $g(x) = 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$ 为 (15, 5) 循环码的码生成多项式。

- ① 画出编码电路；
- ② 写出该码的生成矩阵 $\mathbf{G}$ ；
- ③ 当信息多项式 $m(x) = x^4 + x + 1$ 时，求码多项式及码字；
- ④ 求出该码的一致校验多项式 $h(x)$ 。

10-14 循环码的生成多项式 $g(x) = x^8 + x^7 + x^6 + x^4 + 1$ ，问

$V(x) = x^{14} + x^5 + x + 1$ 是否是码多项式？若不是，求其伴随式，即如何使 $V(x)$ 变为码多项式？

10-15 已知某汉明码的校验矩阵

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

- ① 试求此码的生成矩阵；
- ② 当输入序列为 **110101101010** 时，求编码器的输出序列；

③ 利用 $\mathbf{H}$ 作生成矩阵产生此码的对偶码（要求是系统码）。

10-16 已知一个 (6, 3) 线性分组码的全部码字为

```

1 1 0 1 0 0
1 1 0 0 1 1
0 1 1 0 1 0
0 1 1 1 0 1
1 0 1 0 0 1
0 0 0 1 1 1
1 0 1 1 1 0
0 0 0 0 0 0

```

求该码的生成矩阵和校验矩阵，并讨论其纠检错能力。

10-17 已知 (6, 3) 分组码的一致监督码方程组为

$$\begin{cases} c_5 + c_4 + c_1 + c_0 = 0 \\ c_5 + c_3 + c_1 = 0 \\ c_4 + c_3 + c_2 + c_1 = 0 \end{cases}$$

① 写出相应的一致监督矩阵 $\mathbf{H}$;

② 变换该矩阵为典型阵。

10-18 已知 (7, 3) 分组码的监督关系式为

$$\begin{cases} C_6 + C_3 + C_2 + C_1 = 0 \\ C_6 + C_2 + C_1 + C_0 = 0 \\ C_6 + C_5 + C_1 = 0 \\ C_6 + C_4 + C_0 = 0 \end{cases}$$

求其监督矩阵、生成矩阵，写出全部码字，求其纠错能力。

10-19 若已知监督位 $r = 4$ ，汉明码的长度 n 应为多少？编码效率为多少？写出此汉明码的一致校验矩阵 $\mathbf{H}$ 和生成矩阵 $\mathbf{G}$ 。

10-20 已知 $x^{15} + 1 = (x+1)(x^2 + x+1)(x^4 + x+1)(x^4 + x^3 + 1)$

$(x^4 + x^3 + x^2 + x + 1)$ ，写出所有能构成 $(15, 10)$ 循环码的 $g(x)$ ，并写出对应生成矩阵。

10-21 一个 $(15, 4)$ 循环码的生成多项式 $g(x) = x^{11} + x^{10} + x^6 + x^5 + x + 1$ ，

- ① 求此码的校验多项式 $h(x)$ ；
- ② 求此码的生成矩阵（系统码与非系统码形式） G 。

10-22 一个码长为 $n=15$ 的汉明码，监督位 r 应为多少？编码效率为多少？

10-23 $(3, 1, 3)$ 卷积码编码器框图如图 E10-1 所示。设移位寄存器初始状态是全 0 。当输入序列为 $10110\dots$ 时。

- ① 写出输出序列；
- ② 画出其树状图。

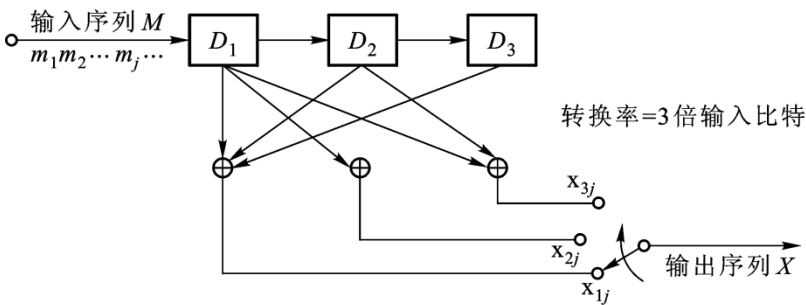


图 E10-1

10-24 一个 $(3, 1, 2)$ 卷积码，其中 $g_1(x) = 1 + x + x^2$ ， $g_2(x) = 1 + x + x^2$ ， $g_3(x) = 1 + x^2$ ，画出该码的编码器框图、码树、网格图和状态转移图。

10-25 卷积编码器如图 10-6-2 所示，卷积编码器的头 13 个输入符号由多项式 $w(x) = 1 + x^2 + x^3 + x^5 + x^7 + x^{11} + x^{12}$ 给出。

- ① 由图 10-6-2 求编码器的输出；
- ② 利用编码器的状态转移图求编码器的输出；

③ 利用网格图求编码器的输出;

④ 直接利用码的生成多项式求编码器的输出。

⑤ 求对应输入多项式 $w(x) = 0 + 0x + 0x^2 + L$ 的编码器输出序列与对应

输入多项式 $w(x) = 1 + 0x + 0x^2 + L$ 的编码器输出序列间的汉明距离。

10-26 某卷积码由下列生成多项式所定义

$$g_1(x) = 1 + x + x^2 + x^3 + x^4$$

$$g_2(x) = 1 + x + x^3 + x^4$$

$$g_3(x) = 1 + x^2 + x^4$$

① 求编码的约束长度;

② 画出用移位寄存器实现的编码器图;

③ 求该码网格图中的状态数;

④ 求对应输入序列 $w(x) = 1 + x + x^2 + x^5 + x^7 + x^{11}$ 的编码器输出序列。

10-27 什么是编码调制? 它有哪些特点?

10-28 通过查表的方法构造一个码长为 31, 能纠正 3 位错误的 BCH 码, 写出其生成多项式。